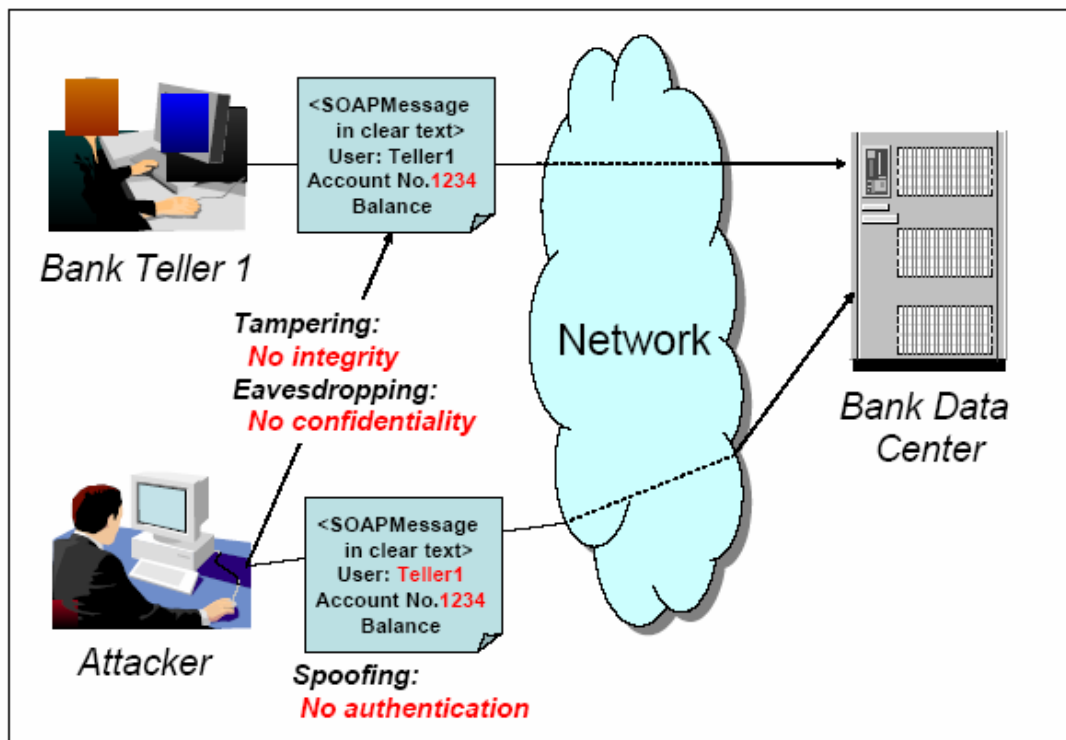


บทที่ 3

Web Service Security (WS-Security)

3.1. Web Service Security (WS-Security)

WS-Security ออกแบบมาเพื่อเพิ่มความปลอดภัยของข้อมูลในระดับ Message ในการส่งข้อมูลระหว่าง เว็บเซอร์วิส แบบ End-to-End และเพิ่มประสิทธิภาพในการส่ง SOAP Message ให้มีความน่าเชื่อถือ มีความถูกต้อง และ มีความปลอดภัย (integrity, confidentiality, authentication) ทำให้ข้อมูลมีความปลอดภัยตลอดเวลาไม่ว่าจะถูกส่งไปให้ Service ไหนก็ตาม



รูปที่ 3.1 แสดงตัวอย่างการส่งข้อมูลผ่านทาง เว็บเซอร์วิส ที่ไม่มีการจัดการเรื่อง Security จากรูปสังเกตได้ว่าสามารถเกิดความเสี่ยงในการที่จะถูกขโมยหรือแก้ไขข้อมูลได้ 3 อย่าง

1. Spoofing: No Authentication

บุคคลที่ 3 สามารถดักจับ SOAP และ แก้ไขข้อมูล SOAP แล้วส่งให้ Service Provider พร้อมทั้ง สามารถปลอมเป็นเจ้าหน้าที่ของธนาคาร ได้

** แก้ไขได้โดยการใช้การพิสูจน์ตัวตน (Authentication)

2. Tempering: No Integrity

บุคคลที่ 3 สามารถดัก SOAP Packet ระหว่าง เว็บเซอร์วิส Client กับ เว็บเซอร์วิส Server ได้ สามารถแก้ไขข้อมูล บัญชีธนาคาร, รหัส ATM, ตัวเลขเงินในบัญชี เว็บเซอร์วิส Server ไม่มีการตรวจสอบว่า ข้อมูลที่ได้รับมานั้น ถูกต้องจริงหรือไม่

** แก้ไขได้โดยการใช้การรักษาความสมบูรณ์ของข้อมูล (Integrity)

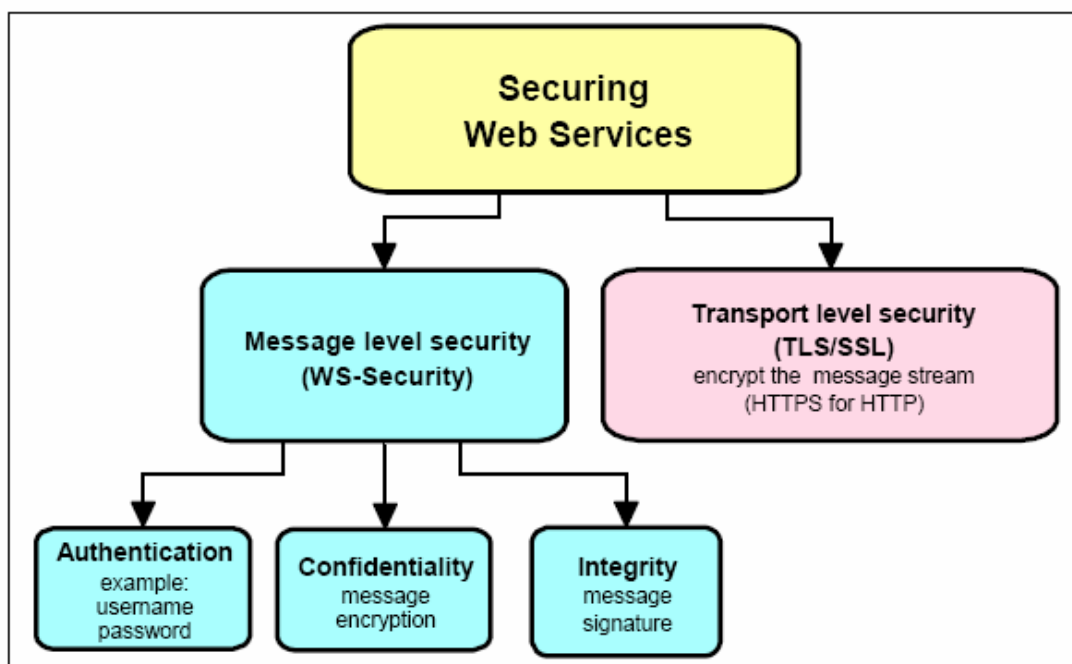
3. Eavesdropping: No Confidentiality

บุคคลที่ 3 สามารถดัก SOAP Packet และ ขโมยข้อมูลไปได้ทั้งหมด เนื่องจากข้อมูลไม่มีการเข้ารหัส ทำให้ข้อมูลที่เป็นความลับของลูกค้าถูกขโมยได้

** แก้ไขได้โดยการใช้การเข้ารหัสข้อมูลเข้ามาช่วย (Encryption)

จากตัวอย่างเหตุการณ์ที่เกิดขึ้นทำให้มีการป้องกัน Web Service Environment ดังนี้

- Message Level Security : Web Service Security (WS-Security)
- Transport Level Security : TLS/SSL



รูปที่ 3.2 แสดงโมเดล Securing Web Services

3.2. การเพิ่มความปลอดภัยในการพิสูจน์ตัวตน (Authentication)

การเพิ่มความปลอดภัยในการพิสูจน์ตัวตนคือขั้นตอนการยืนยันความถูกต้องของหลักฐาน (Identity) ที่แสดงว่าเป็นบุคคลที่กล่าวอ้างจริง การพิสูจน์ตัวตนมีหลายแบบ ไม่ว่าจะเป็น Username Token , digital signature ,ID Assertion , LTPA หรือ Token อื่นๆ ในที่นี้จะกล่าวถึง Username Token โดยเป็นการส่ง username password ไปกับ SOAP ด้วยเพื่อพิสูจน์ตัวตนของผู้ส่ง เมื่อ เว็บ

เซิร์ฟเวอร์ Server ได้รับ username และ password แล้วจะทำการตรวจสอบว่าตรงกับข้อมูลที่มีอยู่หรือไม่ ถ้าสิ่งที่ได้รับมาถูกต้อง Server จะรับ Message และ ทำงานต่อไป

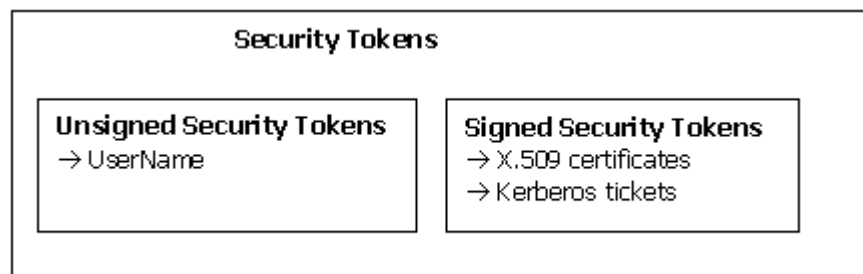
Token แบ่งเป็น 2 รูปแบบ คือ

Unsigned Security Token

เป็น Token ที่ไม่มีผู้รับรอง เป็นการแสดงสิทธิของตนเองให้เห็นเท่านั้น ไม่มีการรับรองแต่อย่างใด เช่น Username Token เป็นต้น

Signed Security Token

เป็น Token ที่มีการรับรองโดยผู้ที่มีความน่าเชื่อถือ เช่น X509 Certificate หรือ Kerberos Ticket เป็นต้น

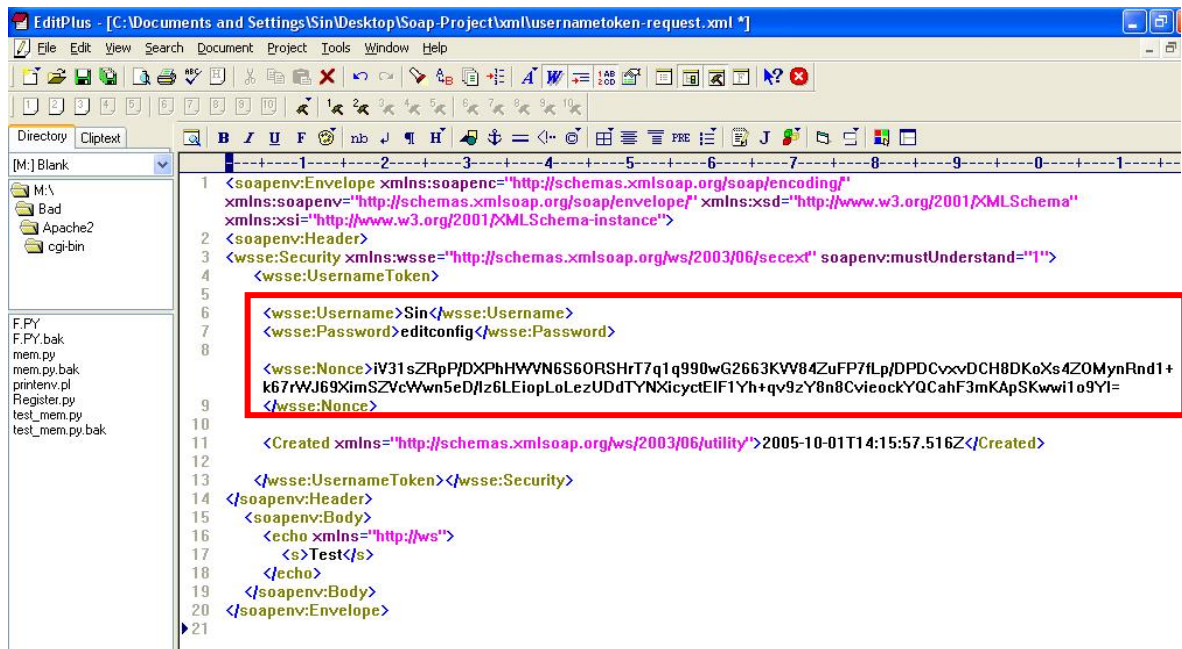


รูปที่ 3.3 แสดงประเภทของ Token

หลักการทำงานของ Username Token

Username Token แบบใช้ Nonce เป็นวิธีที่มีประสิทธิภาพมากวิธีหนึ่งในการตอบโต้การโจมตี Message โดยจะสร้าง cryptographic token แบบสุ่มเพื่อหลีกเลี่ยงการขโมย username token ใน SOAP Message โดยใช้คู่กับ ฟังก์ชัน basicauth แต่ Server จะต้องมีการมี cache ที่มีขนาดใหญ่ไว้ด้วย โดยจะนำมาใช้งานร่วมกับ timestamp ซึ่งจะช่วยให้ Server สามารถจำกัด cache ของ Nonce ให้อยู่ในช่วง “freshness”

เมื่อมีการส่ง Username token จากระบบหนึ่งไปอีกระบบหนึ่งโดยไม่มีการรักษาความปลอดภัยในการขนส่งข้อมูลเช่น HTTP อาจทำให้ token ถูกดักจับและถูกโจมตีกลับได้ key ที่ส่งไปอาจถูกนำกลับมาใช้ใหม่ทำให้เป็นจุดอ่อนในการโจมตีระบบได้ เว้นแต่จะใช้ XML digital signature หรือ XML Encryption เข้ามาช่วย



รูปที่ 3.4 แสดงตัวอย่าง SOAP ที่มีการทำ Username Token แล้ว

3.3. การรักษาความสมบูรณ์ของข้อมูล (Integrity)

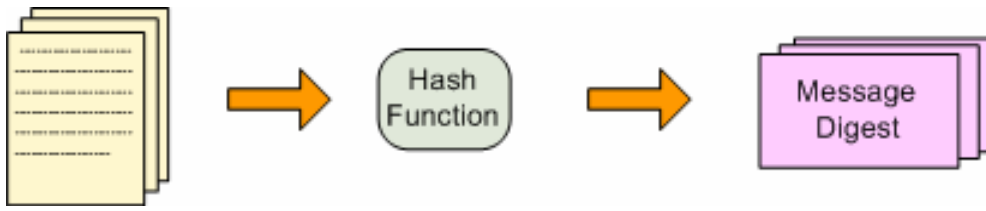
การรักษาความสมบูรณ์ คือการรับรองว่าข้อมูลจะไม่ถูกเปลี่ยนแปลงหรือทำลายไปจากต้นฉบับ ไม่ว่าจะเป็นโดยบังเอิญหรือคัดแปลงโดยเจตนาที่อาจส่งผลเสียต่อข้อมูล การคุกคามความสมบูรณ์ของข้อมูลคือการที่บุคคลที่ไม่ได้รับอนุญาตสามารถที่จะเข้าควบคุมการจัดการของข้อมูลได้ ในที่นี้จะกล่าวถึงการทำให้ XML digital signature บน SOAP Message ซึ่งเป็นการทำลายเซ็นไว้บน SOAP Message ซึ่งถ้า SOAP Message ถูกแก้ไขระหว่างที่ส่งข้อมูล ข้อมูลที่ถูกส่งไปจะทำให้กับผู้รับรู้ทันทีว่าถูกแก้ไขมาแล้ว

หลักการทางาน และตัวอย่าง XML Digital Signature

XML Digital Signature เป็นการระบุลายเซ็นดิจิทัลลงบน XML โดยใช้หลักการ Cryptographic ที่มีอยู่แล้ว เช่น RSA, HMAC, SHA1 ช่วยในการทำให้ผู้รับ Message สามารถมั่นใจได้ว่าไม่มีการขโมยข้อมูลหรือข้อผิดพลาดใดๆเกิดขึ้นใน Message หลังจากที่ได้มีการ sign key ไปแล้ว หรือเรียกได้ว่าเป็น proof of possession

Digital Signature เป็นการนำหลักการของการทำงานของระบบการเข้ารหัสแบบใช้คู่รหัสกุญแจเพื่อการพิสูจน์ตัวตนมาประยุกต์ใช้ ระบบของลายเซ็นดิจิทัลสามารถแบ่งเป็นขั้นตอนได้ดังนี้

1. เมื่อผู้ใช้ต้องการจะส่งข้อมูลไปยังผู้รับ ข้อมูลนั้นจะถูกนำไปเข้าฟังก์ชันทางคณิตศาสตร์ที่เรียกว่า "แฮชฟังก์ชัน" ได้เมสเชสไดเจสต์ (Message Digest) ออกมา



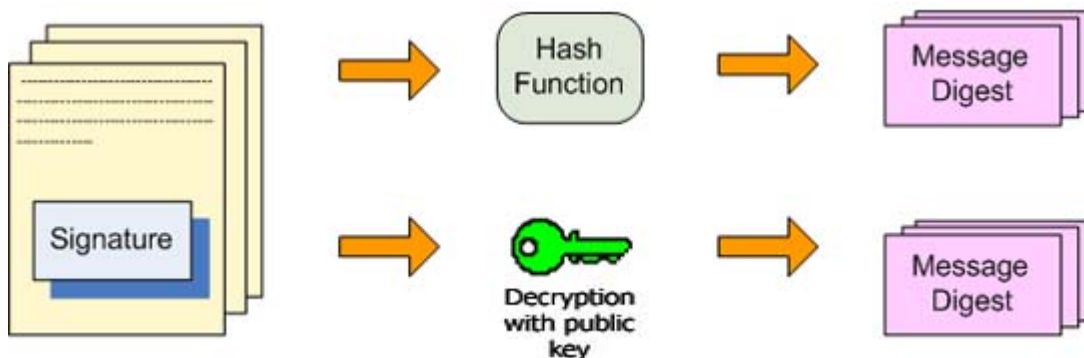
รูปที่ 3.5 การส่งข้อมูลเข้าไปใน Hash function

2. การใช้กุญแจส่วนตัวเข้ารหัสข้อมูล หมายถึงว่าผู้ส่งได้ลงลายเซ็นดิจิทัล ยินยอมที่จะให้ผู้รับ สามารถทำการตรวจสอบด้วยกุญแจสาธารณะของผู้ส่งเพื่อพิสูจน์ตัวตนของผู้ส่งได้



รูปที่ 3.6 การเข้ารหัสเมสเสจไดเจสต์ด้วยกุญแจส่วนตัวเพื่อเป็นการลงลายเซ็น

3. การตรวจสอบข้อมูลว่าถูกส่งมาจากผู้ส่งคนนั้นจริงในด้านผู้รับ โดยการนำข้อมูลมาผ่านแฮชฟังก์ชันเพื่อคำนวณค่าเมสเสจไดเจสต์ และถอดรหัสลายเซ็นอิเล็กทรอนิกส์ด้วยกุญแจสาธารณะของผู้ส่ง ถ้าสามารถถอดได้อย่างถูกต้อง จะเป็นการยืนยันข้อมูลจากผู้ส่งคนนั้นจริง และถ้าข้อมูลเมสเสจไดเจสต์ที่ได้จากการถอดรหัสเท่ากับค่าเมสเสจไดเจสต์ในตอนต้นที่ทำการคำนวณได้ จะถือว่าข้อมูลดังกล่าวนั้นถูกต้อง



รูปที่ 3.7 ขั้นตอนการเปรียบเทียบความถูกต้อง

ลายเซ็นอิเล็กทรอนิกส์นิยมนำไปใช้ในระบบรักษาความปลอดภัยในการชำระเงินผ่านระบบอินเทอร์เน็ต ซึ่งในปัจจุบันนี้การทำธุรกรรมการเงินอิเล็กทรอนิกส์ได้รับความนิยมเป็นอย่างมาก

```

1 <soapenv:Envelope xmlns:soapenc="http://schemas.xmlsoap.org/soap/encoding/"
  xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" >
2 <soapenv:Header>
3 <wsse:Security xmlns:wsse="http://schemas.xmlsoap.org/ws/2003/06/secext"
  soapenv:actor="myActorURI" soapenv:mustUnderstand="1">
4 <wsse:BinarySecurityToken xmlns:wsu="http://schemas.xmlsoap.org/ws/2003/06/utility"
  EncodingType="wsse:Base64Binary" ValueType="wsse:X509v3"
  wsu:Id="wssecurity_binary_security_token_id_408039426707555435" >
5 MIIDQTCCAqgAwIBAgICAQqwDQYJKoZIhvcNAQEFBQAwTjELMAkGA1UEBhMCSEAETAP
6 BgNVBAGTCETHbmFnYXdhMQwwCgYDVQQKEwNjQk0xDDAKBgNVBAsTA1RSTDEQMA4GA1UE
7 AxMHSW50IENBMjAeFw0wMTEwMDEwOTU0MDZaFw0xMTEwMDEwOTU0MDZaMFQxCzAJBgNV
8 BAYTAkpQMREwDwYDVQQLEwhlYW5hZ2F3YTEMAAGGA1UEChMDSUJNMQwwCgYDVQQLEwNU
9 UkwxPjAUBgNVBAMTDVNPQVBSZXI1ZXN0ZlwgZ8wDQYJKoZIhvcNAQEBBQADgY0AMIGJ
10 </wsse:BinarySecurityToken>
11 <Signature xmlns="http://www.w3.org/2000/09/xmldsig#" >
12 <SignedInfo>
13 <CanonicalizationMethod
14 Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#" </CanonicalizationMethod>
15 <SignatureMethod
16 Algorithm="http://www.w3.org/2000/09/xmldsig#rsa-sha1" </SignatureMethod>
17 <Reference URI="#wssecurity_body_id_4508207003845519626" >
18 <Transforms>
19 <Transform Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#" >
20 <ec:InclusiveNamespaces xmlns:ec="http://www.w3.org/2001/10/xml-exc-c14n#"
21 PrefixList="xsi soapenc xsd #default wsu soapenv" </ec:InclusiveNamespaces>
22 </Transform>
23 </Transforms>
24 <DigestMethod Algorithm="http://www.w3.org/2000/09/xmldsig#sha1" </DigestMethod>
25 <DigestValue>zp0qjAnV9B9kA61Si6BZaOqprfQ= </DigestValue>
26 </Reference>
27 </SignedInfo>
28 <SignatureValue>
29 g1ErZ1zUQMjX14OLktdrLlpfAwi7XrJes9GoXaLnT7zenPRjrHMGmNnLfn5
30 a/VV0SqTYfhX6vJMukPFui83jTVvdzPeT8/BoOFwkr/Wsr8VdKS5fuiXcPd9
31 Vc0RcC+zKGV2rfcS33+EbVXaxKVhSS6147ipGU6ENaueMXDzWT8=
32 </SignatureValue>
33 <KeyInfo>
34 <wsse:SecurityTokenReference>
35 <wsse:Reference
36 URI="#wssecurity_binary_security_token_id_408039426707555435" </wsse:Reference>
37 </wsse:SecurityTokenReference>
38 </KeyInfo>
39 </Signature>
40 </wsse:Security>
41 </soapenv:Header>
42 <soapenv:Body xmlns:wsu="http://schemas.xmlsoap.org/ws/2003/06/utility"
  wsu:Id="wssecurity_body_id_4508207003845519626" ><echo xmlns="http://ws/"><s>Test
  Message</s></echo>
43 </soapenv:Body>
44 </soapenv:Envelope>

```

รูปที่ 3.8 แสดงตัวอย่าง SOAP Message ที่มีการใช้ Digital signature

3.4. การเพิ่มความปลอดภัยของข้อมูลที่เป็นความลับ (Confidentiality)

การเพิ่มความปลอดภัยของข้อมูลที่เป็นความลับ คือการรับรองว่าจะมีการเก็บข้อมูลไว้เป็นความลับ และ ผู้มีสิทธิเท่านั้นจึงจะเข้าถึงข้อมูลนั้นได้ ในที่นี้จะกล่าวถึงการทำ XML Encryption บน SOAP Message หรือการเข้ารหัสข้อมูล

หลักการทำงานของ XML Encryption

การทำ XML Encryption คล้ายกับการเข้ารหัสแบบต่างๆ ไป แต่แตกต่างที่ การเข้ารหัส XML สามารถเลือกทำการ เข้ารหัสได้ เฉพาะส่วน หรือเฉพาะ element ได้

3.4.1. อัลกอริทึมในการเข้ารหัสข้อมูลมี 2 ประเภทหลัก คือ

3.4.1.1. อัลกอริทึมแบบสมมาตร (Symmetric key algorithms)

อัลกอริทึมแบบนี้จะใช้กุญแจที่เรียกว่า กุญแจลับ (Secret key) ซึ่งมีเพียงหนึ่งเดียวเพื่อใช้ในการเข้ารหัสและถอดรหัสข้อความที่ส่งไป อัลกอริทึมยังสามารถแบ่งย่อยออกเป็น 2 ประเภท ได้แก่ แบบบล็อก (Block Algorithms) ซึ่งจะทำการเข้ารหัสทีละบล็อก (1 บล็อกประกอบด้วยหลายไบต์ เช่น 64 ไบต์ เป็นต้น) และแบบสตรีม (Stream Algorithms) ซึ่งจะทำการเข้ารหัสทีละไบต์ อัลกอริทึมแบบนี้จะใช้กุญแจที่เรียกว่า กุญแจลับ (Secret key) ซึ่งมีเพียงหนึ่งเดียวเพื่อใช้ในการเข้ารหัสและถอดรหัสข้อความที่ส่งไป อัลกอริทึมยังสามารถแบ่งย่อยออกเป็น 2 ประเภท ได้แก่ แบบบล็อก (Block Algorithms) ซึ่งจะทำการเข้ารหัสทีละบล็อก (1 บล็อกประกอบด้วยหลายไบต์ เช่น 64 ไบต์ เป็นต้น) และแบบสตรีม (Stream Algorithms) ซึ่งจะทำการเข้ารหัสทีละไบต์

3.4.1.2. อัลกอริทึมแบบอสมมาตร (Asymmetric key algorithms)

อัลกอริทึมนี้จะใช้กุญแจสองตัวเพื่อทำงาน ตัวหนึ่งใช้ในการเข้ารหัสและอีกตัวหนึ่งใช้ในการถอดรหัสข้อมูลที่เข้ารหัสมาโดยกุญแจตัวแรก อัลกอริทึมกลุ่มสำคัญในแบบอสมมาตรนี้คือ อัลกอริทึมแบบกุญแจสาธารณะ (Public keys Algorithms) ซึ่งใช้กุญแจที่เรียกกันว่า กุญแจสาธารณะ (Public keys) ในการเข้ารหัสและใช้กุญแจที่เรียกกันว่า กุญแจส่วนตัว (Private keys) ในการถอดรหัสข้อมูลนั้น กุญแจสาธารณะนี้สามารถส่งมอบให้กับผู้อื่นได้ เช่น เพื่อนร่วมงานที่เราต้องการติดต่อดู หรือแม้กระทั่งวางไว้บนเว็บไซต์เพื่อให้ผู้อื่นสามารถดาวน์โหลดไปใช้งานได้ สำหรับกุญแจส่วนตัวนั้นต้องเก็บไว้กับผู้ใช้เป็นเจ้าของกุญแจส่วนตัวเท่านั้นและห้ามเปิดเผยให้ผู้อื่นทราบโดยเด็ดขาด

3.4.2. ปัญหาของอัลกอริทึมแบบสมมาตร (Symmetric key Problem)

อัลกอริทึมแบบสมมาตรมีความสำคัญไม่ด้อยไปกว่าอัลกอริทึมแบบอสมมาตร ทั้งนี้เนื่องจากอัลกอริทึมแบบแรกทำงานได้รวดเร็วและง่ายต่อการใช้งานกว่าแบบหลัง อย่างไรก็ตาม อัลกอริทึมแบบสมมาตรยังมีปัญหาที่สำคัญ 3 ประการ ซึ่งเป็นข้อจำกัดในการใช้งานอัลกอริทึมนี้

1. ในการใช้งานอัลกอริทึมนี้ สองกลุ่มที่ต้องการแลกเปลี่ยนข้อมูลกัน (เช่น องค์กร ก และ ข) จำเป็นต้องแลกเปลี่ยนกุญแจลับกันก่อน (ซึ่งอาจหมายถึงส่งมอบกุญแจลับให้กับอีกกลุ่มหนึ่ง) การแลกเปลี่ยนกุญแจนั้นอาจทำได้อย่างยุ่งยากและไม่สะดวก

2. ทั้งสองกลุ่มต้องรักษากุญแจลับนั้นไว้เป็นอย่างดี ห้ามเปิดเผยให้ผู้อื่นล่วงรู้โดยเด็ดขาด การที่กุญแจถูกเปิดเผยออกไปสู่ผู้อื่น (จะโดยกลุ่มใดกลุ่มหนึ่งก็ตาม) และอีกกลุ่มหนึ่งไม่ได้รับทราบปัญหานี้ อาจก่อให้เกิดปัญหากับกลุ่มที่ไม่ทราบนี้ได้ เช่น กลุ่มนี้อาจส่งข้อความที่เป็นความลับไปให้กับอีกกลุ่มหนึ่ง แต่ข้อความนี้อาจถูกเปิดเผยได้โดยใช้กุญแจลับที่ล่วงรู้โดยผู้อื่น

3. สำหรับสองกลุ่มที่ต้องการติดต่อกัน จำเป็นต้องใช้กุญแจลับเป็นจำนวน 1 กุญแจเพื่อติดต่อกัน สมมติว่ามีผู้ที่ต้องติดต่อกันเป็นจำนวน n กลุ่ม จำนวนกุญแจลับทั้งหมดที่ต้องแลกเปลี่ยน

กันคิดเป็นจำนวนทั้งหมด C_{2n} หรือเท่ากับ $n(n-1)/2$ กุญแจ ซึ่งจะเห็นได้ว่าจำนวนกุญแจมีมากมายเกินไป ซึ่งอาจก่อให้เกิดปัญหาด้านการรักษาความปลอดภัยให้กับกุญแจเหล่านี้

อัลกอริทึมแบบกุญแจสาธารณะ (ซึ่งเป็นแบบอสมมาตร) ช่วยแก้ปัญหาเหล่านี้ได้ทั้งหมด ผู้ใช้ที่ถือกุญแจส่วนตัวและต้องการให้บุคคลอื่นที่ตนติดต่อด้วยส่งเอกสารหรือข้อความที่เข้ารหัสมาหาตน สามารถเผยแพร่กุญแจสาธารณะของตนไว้บนเว็บไซต์หรือในที่สาธารณะซึ่งผู้อื่นสามารถเข้ามาดาวน์โหลดไปใช้งานได้ วิธีการใช้งานคือให้บุคคลอื่นที่มาดาวน์โหลดกุญแจไปนั้นทำการเข้ารหัสข้อความที่ต้องการส่งด้วยกุญแจสาธารณะ แล้วจึงส่งข้อความที่เข้ารหัสไปให้กับผู้เป็นเจ้าของกุญแจสาธารณะ โดยวิธีนี้จะไม่มีผู้อื่นสามารถเปิดดูข้อความที่เข้ารหัสนั้นได้ยกเว้นผู้ที่ถือกุญแจส่วนตัว (ที่เป็นคู่ของกุญแจสาธารณะนั้น) จึงจะสามารถเปิดข้อความนี้ดูได้

การเผยแพร่กุญแจสาธารณะในสถานที่ต่างๆ ได้ทำให้ลดความยุ่งยากในการแลกเปลี่ยนกุญแจกันซึ่งเป็นปัญหาข้อแรกของการเข้ารหัสแบบสมมาตร สำหรับปัญหาที่ว่าทั้งสองกลุ่มจะต้องรักษากุญแจลับไว้เป็นอย่างดีนั้น วิธีการของกุญแจสาธารณะจะทำให้ผู้ที่ต้องรับผิดชอบเหลือเพียงผู้เดียว กล่าวคือ ผู้ถือกุญแจส่วนตัว ซึ่งห้ามให้ผู้อื่นล่วงรู้โดยเด็ดขาด

สำหรับปัญหาที่สามที่ว่าจำนวนกุญแจลับที่จำเป็นต้องใช้มีมากมายเกินไป วิธีการของกุญแจสาธารณะจะใช้จำนวนกุญแจที่ประหยัดกว่า เนื่องจากกุญแจสาธารณะ 1 กุญแจของกลุ่มๆ หนึ่งจะสามารถเผยแพร่ให้กับทั้งกลุ่มก็ได้ที่เราต้องการติดต่อด้วย (แทนที่จะเป็น 1 กุญแจลับต่อสองกลุ่มที่ต้องการติดต่อกัน) ดังนั้นถ้ามีกลุ่มที่ต้องติดต่อกันจำนวน n กลุ่ม จำนวนกุญแจส่วนตัวที่ต้องระวังก็น่าจะเป็น n กุญแจ ซึ่งจะเห็นได้ว่าลดลงไปได้เป็นจำนวนมาก

ข้อเสียที่สำคัญของระบบกุญแจสาธารณะที่สำคัญคือ ต้องใช้เวลาในการคำนวณการเข้ารหัสและถอดรหัส เมื่อเทียบกับระบบกุญแจสมมาตร และอาจใช้เวลาเป็นพันเท่าของเวลาที่ใช้โดยระบบกุญแจสมมาตร

3.4.3. ความแข็งแกร่งของอัลกอริทึมสำหรับการเข้ารหัส

ความแข็งแกร่งของอัลกอริทึมหมายถึงความยากในการที่ผู้บุกรุกจะสามารถถอดรหัสข้อมูลได้โดยปราศจากกุญแจที่ใช้ในการเข้ารหัส ซึ่งจะขึ้นอยู่กับปัจจัยดังนี้

1. การเก็บกุญแจเข้ารหัสไว้อย่างเป็นความลับ ผู้เป็นเจ้าของกุญแจลับหรือส่วนตัวต้องระมัดระวังไม่ให้กุญแจสูญหายหรือล่วงรู้โดยผู้อื่น

2. ความยาวของกุญแจเข้ารหัส ปกติกุญแจเข้ารหัสจะมีความยาวเป็นบิต ยิ่งจำนวนบิตของกุญแจยิ่งมาก ยิ่งทำให้การเดาเพื่อค้นหากุญแจที่ถูกต้องเป็นไปได้ยากยิ่งขึ้น (เช่น กุญแจขนาด 1 บิตจะสามารถแทนตัวเลขได้ 2 ค่าคือ 0 กับ 1 กุญแจขนาด 2 บิต จะเป็นไปได้ 4 ค่าคือ 0, 1, 2, 3 เป็นต้น)

3. ความไม่เกรงกลัวต่อการศึกษาหรือดักถอดรหัสมือเพื่อหารูปแบบของการเข้ารหัส อัลกอริทึมที่ดีต้องเปิดให้ผู้รู้ทำการศึกษาในรายละเอียดได้โดยไม่เกรงว่าผู้ศึกษาจะสามารถจับรูปแบบของการเข้ารหัสได้

4. การมีประตูลับในอัลกอริทึม อัลกอริทึมที่ดีต้องไม่แฝงไว้ด้วยประตูลับที่สามารถใช้เป็นทางเข้าไปสู่อัลกอริทึม แล้วอาจใช้เพื่อทำการถอดรหัสข้อมูลได้ ประตูลับนี้ทำให้ไม่จำเป็นต้องใช้กุญแจในการถอดรหัส

5. ความไม่เกรงกลัวต่อปัญหาการหาความสัมพันธ์ในข้อมูลที่ได้รับ กล่าวคือเมื่อผู้บุกรุกทราบข้อมูลบางอย่างที่เป็นข้อมูลตั้งต้นซึ่งยังไม่ได้เข้ารหัส รวมทั้งมีข้อมูลที่เข้ารหัสแล้ว (ของข้อมูลตั้งต้นนั้น) ผู้บุกรุกอาจจะสามารถหาความสัมพันธ์ระหว่างข้อความทั้งสองนั้นได้ ซึ่งจะเป็นวิธีการในการถอดรหัสข้อมูลได้ ปัญหานี้เรียกกันว่า Known plaintext attack (คำว่า plaintext หมายถึงข้อความตั้งต้นที่ยังไม่ได้ผ่านการเข้ารหัส)

คุณสมบัติของข้อความตั้งต้น คุณสมบัตินี้อาจใช้เป็นช่องทางในการถอดรหัสข้อมูล อัลกอริทึมที่ดีต้องไม่ใช้คุณสมบัติของข้อความเป็นกลไกในการเข้ารหัสข้อมูล

คำแนะนำในการเลือกใช้อัลกอริทึมคือให้ใช้อัลกอริทึมที่ได้มีการใช้งานมาเป็นระยะเวลานานแล้ว ทั้งนี้เนื่องจากหากปัญหาของอัลกอริทึมนี้มีจริง ก็คงเกิดขึ้นมานานแล้วและก็คงเป็นที่ทราบกันแล้ว นั่นคืออย่างน้อยที่สุดจวบจนกระทั่งถึงปัจจุบัน ก็ยังไม่มีการบุกรุกที่ทำให้อัลกอริทึมนั้นไม่สามารถใช้งานได้อย่างปลอดภัยเป็นที่ประจักษ์ ดังนั้นจึงไม่ควรใช้อัลกอริทึมใหม่ๆ ที่เพิ่งได้มีการนำเสนอกันสู่สาธารณะ เพราะอาจมีช่องโหว่แฝงอยู่และยังไม่เป็นที่ทราบในขณะนี้

3.4.4. ความยาวของกุญแจที่ใช้ในการเข้ารหัส

ความยาวของกุญแจเข้ารหัสมีหน่วยนับเป็นบิต หนึ่งบิตในคอมพิวเตอร์เป็นตัวเลขฐานสองที่ประกอบด้วยค่า 0 และ 1 กุญแจที่มีความยาว 1 บิต ตัวเลขที่เป็นไปได้เพื่อแทนกุญแจนั้น จึงอาจมีค่าเป็น 0 หรือ 1 กุญแจที่มีความยาว 2 บิต ตัวเลขที่เป็นไปได้จึงเป็น 0, 1, 2 และ 3 ตามลำดับ กุญแจที่มีความยาว 3 บิต ตัวเลขที่เป็นไปได้จะอยู่ระหว่าง 0 ถึง 7 ดังนั้นเมื่อเพิ่มความยาวของกุญแจทุกๆ 1 บิต ค่าที่เป็นไปได้ของกุญแจจะเพิ่มขึ้นเป็นสองเท่าตัว หรือจำนวนกุญแจที่เป็นไปได้จะเพิ่มขึ้นเป็น 2 เท่าตัวนั่นเอง

ฉะนั้นจะเห็นได้ว่ากุญแจยิ่งมีความยาวมาก โอกาสที่ผู้บุกรุกจะสามารถคาดเดากุญแจที่ตรงกับหมายเลขที่ถูกต้องของกุญแจจะยิ่งยากมากขึ้นตามลำดับ ในการที่ผู้บุกรุกลองผิดลองถูกกับกุญแจโดยใช้กุญแจที่มีหมายเลขต่างๆ กัน เพื่อหวังที่จะพบกุญแจที่ถูกต้องและสามารถใช้อัดรหัสข้อมูลได้ การลองผิดลองถูกนี้เราเรียกกันว่า Key search หรือการค้นหากุญแจนั่นเอง ทฤษฎีได้กล่าวไว้ว่าการลองผิดลองถูกนี้โดยเฉลี่ยจะต้องทดลองกับกุญแจเป็นจำนวนครึ่งหนึ่งของกุญแจทั้งหมดก่อนที่จะพบกุญแจที่ถูกต้อง

ความยาวของกุญแจที่มีขนาดเหมาะสมจึงขึ้นอยู่กับความเร็วในการค้นหากุญแจของผู้บุกรุกและระยะเวลาที่ต้องการให้ข้อมูลมีความปลอดภัย ตัวอย่างเช่น ถ้าผู้บุกรุกสามารถลองผิดลองถูกกับกุญแจเป็นจำนวน 10 กุญแจภายในหนึ่งวินาทีแล้ว กุญแจที่มีความยาว 40 บิต จะสามารถป้องกันข้อมูลไว้ได้ 3,484 ปี ถ้าผู้บุกรุกสามารถลองได้เป็นจำนวน 1 ล้านกุญแจในหนึ่งวินาที (เทคโนโลยีปัจจุบันสามารถทำได้) กุญแจที่มีความยาว 40 บิตจะสามารถป้องกันข้อมูลไว้ได้เพียง 13 วันเท่านั้น (ซึ่งอาจไม่เพียงพอสำหรับในบางลักษณะงาน) ด้วยเทคโนโลยีในปัจจุบันหากผู้บุกรุกสามารถทดลองได้เป็นจำนวน 1,000 ล้านกุญแจในหนึ่งวินาที กุญแจขนาด 128 บิตจะสามารถป้องกันข้อมูลไว้ได้ 1022 ปี ดังนั้นด้วยลักษณะงานทั่วไปกุญแจขนาด 128 บิตจะพอเพียงต่อการรักษาความลับของข้อมูลเอาไว้ได้

3.4.5. อัลกอริทึมสำหรับการเข้ารหัสแบบสมมาตร

อัลกอริทึมสำหรับการเข้ารหัสแบบสมมาตรในปัจจุบันมีเป็นจำนวนมาก ข้างล่างนี้จะนำเสนอเพียงจำนวนหนึ่งซึ่งเป็นอัลกอริทึมที่เป็นที่รู้จักกันดีในวงการของการเข้ารหัสข้อมูล

อัลกอริทึม DES

DES ย่อมาจาก Data Encryption Standard เป็นอัลกอริทึมแบบบล็อกซึ่งใช้กุญแจที่มีขนาดความยาว 56 บิตและเป็นอัลกอริทึมที่มีความแข็งแกร่ง แต่เนื่องด้วยขนาดความยาวของกุญแจที่มีขนาดเพียง 56 บิต ซึ่งในปัจจุบันถือว่าสั้นเกินไป ผู้บุกรุกอาจใช้วิธีการลองผิดลองถูกเพื่อค้นหากุญแจที่ถูกต้องสำหรับการถอดรหัสได้

อัลกอริทึม Triple-DES

Triple-DES เป็นอัลกอริทึมที่เสริมความปลอดภัยของ DES ให้มีความแข็งแกร่งมากขึ้นโดยใช้อัลกอริทึม DES เป็นจำนวนสามครั้งเพื่อทำการเข้ารหัส แต่แต่ละครั้งจะใช้กุญแจในการเข้ารหัสที่แตกต่างกัน ดังนั้นจึงเปรียบเสมือนการใช้กุญแจเข้ารหัสที่มีความยาวเท่ากับ $56 \times 3 = 168$ บิต Triple-DES ได้ถูก ใช้งานกับสถาบันทางการเงินอย่างแพร่หลาย รวมทั้งใช้งานกับโปรแกรม Secure Shell (ssh) ด้วย

อัลกอริทึม Rijndael (หรืออัลกอริทึม AES)

อัลกอริทึมนี้ได้รับการพัฒนาโดย Joan Daemen และ Vincent Rijmen ในปี 2000 อัลกอริทึมได้รับการคัดเลือกโดยหน่วยงาน National Institute of Standard and Technology (NIST) ของสหรัฐอเมริกาให้เป็นมาตรฐานในการเข้ารหัสชั้นสูงของประเทศ อัลกอริทึมมีความเร็วสูงและมีขนาดกะทัดรัดโดยสามารถใช้กุญแจที่มีความยาวขนาด 128, 192 และ 256 บิต

3.4.6. อัลกอริทึมสำหรับการเข้ารหัสแบบกุญแจสาธารณะ (หรือการเข้ารหัสแบบอสมมาตร)

อัลกอริทึม RSA

อัลกอริทึม RSA ได้รับการพัฒนาขึ้นที่มหาวิทยาลัย MIT ในปี 1977 โดยศาสตราจารย์ 3 คน ซึ่งประกอบด้วย Ronald Rivest, Adi Shamir และ Leonard Adleman ชื่อของอัลกอริทึมได้รับ

การตั้งชื่อตามตัวอักษรตัวแรกของนามสกุลของศาสตราจารย์ทั้งสามคน อัลกอริทึมนี้สามารถใช้ในการเข้ารหัสข้อมูลรวมทั้งการลงลายมือชื่ออิเล็กทรอนิกส์ด้วย

อัลกอริทึม DSS

DSS ย่อมาจาก Digital Signature Standard อัลกอริทึมนี้ได้รับการพัฒนาขึ้นมาโดย National Security Agency ในประเทศสหรัฐอเมริกาและได้รับการรับรองโดย NIST ให้เป็นมาตรฐานกลางสำหรับการลงลายมือชื่ออิเล็กทรอนิกส์ในประเทศสหรัฐอเมริกา

3.4.7. อัลกอริทึมสำหรับสร้างเมสเสจไดเจสต์

เมสเสจไดเจสต์ (Message Digest) หรือเรียกสั้นๆ ว่าไดเจสต์ แปลว่าข้อความสรุปจากเนื้อหาข้อความตั้งต้น โดยปกติข้อความสรุปจะมีความยาวน้อยกว่าความยาวของข้อความตั้งต้นมาก จุดประสงค์สำคัญของอัลกอริทึมนี้คือ การสร้างข้อความสรุปที่สามารถใช้เป็นตัวแทนของข้อความตั้งต้นได้ โดยทั่วไปข้อความสรุปจะมีความยาวอยู่ระหว่าง 128 ถึง 256 บิต และจะไม่ขึ้นกับขนาดความยาวของข้อความตั้งต้น

3.4.8. คุณสมบัติที่สำคัญของอัลกอริทึมสำหรับสร้างไดเจสต์มีดังนี้

ทุกๆ บิตของไดเจสต์จะขึ้นอยู่กับทุกบิตของข้อความตั้งต้น ถ้าบิตใดบิตหนึ่งของข้อความตั้งต้นเกิดการเปลี่ยนแปลง เช่น ถูกแก้ไข ทุกๆ บิตของไดเจสต์จะมีโอกาสร้อยละ 50 ที่จะแปรเปลี่ยนค่าไปด้วย ซึ่งหมายถึงว่า 0 เปลี่ยนค่าเป็น 1 และ 1 เปลี่ยนเป็น 0

คุณสมบัติข้อนี้สามารถอธิบายได้ว่าการเปลี่ยนแปลงแก้ไขข้อความตั้งต้นโดยผู้ไม่ประสงค์ดีแม้ว่าอาจแก้ไขเพียงเล็กน้อยก็ตาม เช่น เพียง 1 บิตเท่านั้น ก็จะส่งผลให้ผู้รับข้อความทราบว่าข้อความที่ตนได้รับไม่ใช่ข้อความตั้งต้น (โดยการนำข้อความที่ตนได้รับเข้าอัลกอริทึมเพื่อทำการคำนวณหาไดเจสต์ออกมา แล้วจึงเปรียบเทียบไดเจสต์ที่คำนวณได้กับไดเจสต์ที่ส่งมาให้ด้วย ถ้าต่างกัน แสดงว่าข้อความที่ได้รับนั้นถูกเปลี่ยนแปลงแก้ไข)

โอกาสที่ข้อความตั้งต้น 2 ข้อความใดๆ ที่มีความแตกต่างกัน จะสามารถคำนวณได้ค่าไดเจสต์เดียวกันมีโอกาสน้อยมาก

คุณสมบัติข้อนี้ทำให้แน่ใจได้ว่า เมื่อผู้ไม่ประสงค์ดีทำการแก้ไขข้อความตั้งต้น ผู้รับข้อความที่ถูกแก้ไขไปแล้วนั้นจะสามารถตรวจพบได้ถึงความผิดปกติที่เกิดขึ้นอย่างแน่นอน

อย่างไรก็ตามในทางทฤษฎีแล้ว มีโอกาสที่ข้อความ 2 ข้อความที่แตกต่างกันจะสามารถคำนวณแล้วได้ค่าไดเจสต์เดียวกัน ปัญหานี้เรียกกันว่าการชนกันของไดเจสต์ (Collision) อัลกอริทึมสำหรับสร้างไดเจสต์ที่ดีควรจะมีโอกาสน้อยมากๆ ที่จะก่อให้เกิดปัญหาการชนกันของไดเจสต์ อัลกอริทึมสำหรับสร้างไดเจสต์ยอดนิยมมีดังนี้

อัลกอริทึม MD2

ผู้พัฒนาคือ Ronald Rivest อัลกอริทึมนี้เชื่อกันว่ามีความแข็งแกร่งที่สุดในบรรดาอัลกอริทึมต่างๆ ที่ Rivest พัฒนาขึ้นมา (ความแข็งแกร่งพิจารณาได้จากคุณสมบัติตามประการ

ข้างต้น) ข้อเสียของอัลกอริทึมนี้คือใช้เวลานานในการคำนวณไคเจสต์หนึ่งๆ MD2 จึงไม่ค่อยได้มีการใช้งานกันมากนัก MD2 สร้างไคเจสต์ที่มีความยาว 128 บิต

อัลกอริทึม MD4

ผู้พัฒนาคือ Rivest เช่นเดียวกับ MD2 อัลกอริทึมนี้พัฒนาขึ้นมาเพื่อแก้ปัญหาค่าซ้ำในการคำนวณของ MD2 อย่างไรก็ตามในภายหลังได้พบว่าอัลกอริทึมมีข้อบกพร่องที่เกี่ยวข้องกับคุณสมบัติข้อที่สามโดยตรง กล่าวคือปัญหาการชนกันของไคเจสต์มีโอกาสเกิดขึ้นได้ ซึ่งผู้บุกรุกอาจใช้ประโยชน์จากจุดอ่อนนี้เพื่อทำการแก้ไขข้อความดั้งเดิมที่ส่งมาให้ได้ MD4 ผลิตไคเจสต์ที่มีขนาด 128 บิต

อัลกอริทึม MD5

Rivest เป็นผู้พัฒนาเช่นกันโดยพัฒนาต่อจาก MD4 เพื่อให้มีความปลอดภัยที่สูงขึ้น ถึงแม้จะเป็นที่นิยมใช้งานกันอย่างแพร่หลาย ทว่าในปี 1996 ก็มีผู้พบจุดบกพร่องของ MD5 (เช่นเดียวกับ MD4) จึงทำให้ความนิยมเริ่มลดลง MD5 ผลิตไคเจสต์ที่มีขนาด 128 บิต

อัลกอริทึม SHA

SHA ย่อจาก Secure Hash Algorithm อัลกอริทึม SHA ได้รับแนวคิดในการพัฒนามาจาก MD4 และได้รับการพัฒนาขึ้นมาเพื่อใช้งานร่วมกับอัลกอริทึม DSS (ซึ่งใช้ในการลงลายมือชื่ออิเล็กทรอนิกส์) หลังจากที่ได้มีการตีพิมพ์เผยแพร่อัลกอริทึมนี้ได้ไม่นาน NIST ก็ประกาศตามมาว่าอัลกอริทึมจำเป็นต้องได้รับการแก้ไขเพิ่มเติมเล็กน้อยเพื่อให้สามารถใช้งานได้เหมาะสม SHA สร้างไคเจสต์ที่มีขนาด 160 บิต

อัลกอริทึม SHA-1

SHA-1 เป็นอัลกอริทึมที่แก้ไขเพิ่มเติมเล็กน้อยจาก SHA การแก้ไขเพิ่มเติมนี้เป็นที่เชื่อกันว่าทำให้อัลกอริทึม SHA-1 มีความปลอดภัยที่สูงขึ้น SHA-1 สร้างไคเจสต์ที่มีขนาด 160 บิต

อัลกอริทึม SHA-256, SHA-384 และ SHA-512

NIST เป็นผู้นำเสนออัลกอริทึมทั้งสามนี้ในปี 2001 เพื่อใช้งานร่วมกับอัลกอริทึม AES (ซึ่งเป็นอัลกอริทึมในการเข้ารหัสแบบสมมาตร) อัลกอริทึมเหล่านี้สร้างไคเจสต์ที่มีขนาด 256, 384 และ 512 บิต ตามลำดับ

นอกจากอัลกอริทึมสำหรับการสร้างไคเจสต์ที่กล่าวถึงไปแล้วนั้น อัลกอริทึมสำหรับการเข้ารหัสแบบสมมาตร เช่น DES สามารถใช้ในการสร้างไคเจสต์เช่นกัน วิธีการใช้งานอัลกอริทึมแบบสมมาตรเพื่อสร้างไคเจสต์คือ ให้เลือกกุญแจลับสำหรับการเข้ารหัสขึ้นมา 1 กุญแจโดยวิธีการเลือกแบบสุ่ม และต่อมาใช้กุญแจนี้เพื่อเข้ารหัสข้อความดั้งเดิม แล้วใช้เฉพาะบล็อกสุดท้ายที่เข้ารหัสแล้วเพื่อเป็นไคเจสต์ของข้อความทั้งหมด (ไม่รวมบล็อกอื่นๆ ที่เข้ารหัสแล้ว) อัลกอริทึมแบบสมมาตรสามารถสร้างไคเจสต์ที่มีคุณภาพดี แต่ข้อเสียคือต้องใช้เวลาในการคำนวณไคเจสต์มาก

ไคเจสต์เป็นเครื่องมือที่สำคัญที่สามารถใช้ในการตรวจสอบว่าไฟล์ในระบบที่ใช้งานมีการเปลี่ยนแปลงแก้ไขหรือไม่ (ไม่ว่าจะโดยเจตนาหรือไม่ก็ตาม) บางครั้งการเปลี่ยนแปลงแก้ไขอาจถูกกระทำโดยผู้ที่ไม่มีสิทธิ์ เช่น ผู้บุกรุก เป็นต้น

วิธีการใช้ไคเจสต์เพื่อตรวจสอบไฟล์ในระบบคือให้เลือกใช้อัลกอริทึมหนึ่ง เช่น MD5 เพื่อสร้างไคเจสต์ของไฟล์ในระบบและเก็บไคเจสต์นั้นไว้อีกที่หนึ่งนอกระบบ ภายหลังจากระยะเวลาหนึ่งที่กำหนดไว้ เช่น 1 เดือน ก็มาคำนวณไคเจสต์ของไฟล์เดิมอีกครั้งหนึ่ง แล้วเปรียบเทียบไคเจสต์ใหม่นี้กับไคเจสต์ที่เก็บไว้นอกระบบว่าตรงกันหรือไม่ ถ้าตรงกัน ก็แสดงว่าไฟล์ในระบบยังเป็นปกติเช่นเดิม

ไคเจสต์ยังเป็นส่วนหนึ่งของการลงลายมือชื่ออิเล็กทรอนิกส์ กล่าวคือการลงลายมือชื่ออิเล็กทรอนิกส์ในปัจจุบันจะทำการลงลายมือชื่อกับไคเจสต์ของข้อความตั้งต้นแทนการลงลายมือชื่อกับข้อความตั้งต้นทั้งข้อความ

การเข้ารหัส XML สามารถแบ่งได้เป็น 3 ส่วนคือ

1. การเข้ารหัส XML แบบทั้งเอกสาร
2. การเข้ารหัสเฉพาะ element ในเอกสาร
3. การเข้ารหัสเฉพาะ ข้อมูลในเอกสาร

การเข้ารหัส XML นี้ทำให้สามารถจัดการได้ว่าข้อมูลไหน สมควรเป็นความลับข้อมูลไหน ควรจัดการอย่างไร มีความยืดหยุ่นสูง และช่วยลดเวลาในการเข้ารหัส

```
1  <?xml version="1.0" encoding="UTF-8"?>
2
3  <EncryptedData xmlns="http://www.w3.org/2001/04/xmldsig#" Id="Test"
   Type="http://www.isi.edu/in-notes/iana/assignments/media-types/text/xml">
4    <EncryptionMethod Algorithm="http://www.w3.org/2001/04/xmldsig#tripledes-cbc" />
5    <ds:KeyInfo xmlns:ds="http://www.w3.org/2000/09/xmldsig#">
6      <KeyName>theKey</KeyName>
7    </ds:KeyInfo>
8    <CipherData>
9      <CipherValue>v0oy2cty7WLj5RDNbVZo3UG/BtTt08IWWXf0Y/EHu/qivSa/p7wXP03mSXvtaGY87+
        hvjCrrH4mp
10     cKr+OxrTJumhVKHyptnHTpJksHxehqM9tCZyQDR0rZeLo76KrU2/FDS2ZDS3SLYDda3PUf5v0paU
11     k2TdoCJIttXPwvbwULpzvP8/5BYVT+t8bqm24YEC9Q8shlmAc/OdXV03zfWlMKoqAZj1je03aImR
12     BobTnqd1o+iilmFUEV6DmgOpdwDuu+Csn9J3wE3C4co6ISH6zEqLFWmJAZwnytGTi6/t24fzRHeK
13     7f+SOIC+7yLjnnHPBfpvp2ciC/IEg5jHd2qg4vhSKN0k6qfJUJpc5M4kl4dE79lv7LpF5m22tf8
14     z1pDXmwgki1YVi3ksIPXmHALil/y4ggx4u2pCAFR17Bk4bxiRL4lNpoJx/ykl8JfidBrGvKiuYB
15     nbdNCn/8jKzRUyyTfcgHQXFCeUe7JrueiQJjiTIPwl3Ye8u4fipDfegCnbV/rb5/a7vFe2Z/Rfrq
16     xUPBJQ9K0uP4+DMTP3qAfKL8F0ggQ5UnvnbLFYNxnx4D</CipherValue>
17   </CipherData>
18 </EncryptedData>
19
20
```

รูปที่ 3.9 แสดงตัวอย่าง SOAP Message มีการเข้ารหัสทั้งเอกสารแบบ triple-DES

3.5. การเพิ่มความปลอดภัยในการขนส่งข้อมูล (Transport – level Security)

Transport – level Security นิยมใช้ในการเพิ่มความปลอดภัยการสื่อสารของ เว็บเซอร์วิส ผ่าน protocol HTTP เช่น Secure Sockets Layer Transport หรือ Layer Security โดย HTTPS จะทำการตกลงระหว่าง Client กับ Server ที่ถูกรับรองด้วย Certificate ซึ่งเป็น Certificate ที่เขียนขึ้นมาเอง หรือ Certificate ที่มีผู้รับรอง ก็ได้ ทำให้สามารถกำหนดได้ว่าจะทำการจัดการกับ security ใน message ได้อย่างอิสระอย่างไรก็ตาม โดย HTTPS จะทำการเข้ารหัส HTTP Message ที่มีการแลกเปลี่ยนกัน ไม่สามารถจัดการ security ได้หมด ยังมีบางส่วนเช่นใน SSL ไม่สามารถจัดการ JMS ได้

ในการใช้ Transport – level Security นิยมใช้ผ่าน HTTPS ซึ่งสามารถใช้ได้โดยแทน https:// ใน port address

3.5.1. หลักการทำงานของ SSL

Secure Sockets Layer (SSL) ใช้ในโพรโตคอลระดับแอปพลิเคชันคือ Hypertext Transfer Protocol (HTTP) ซึ่งเป็นการสื่อสารผ่านเว็บให้ปลอดภัย SSL ทำให้เกิดการสื่อสารอย่างปลอดภัยระหว่างไคลเอนต์และเซิร์ฟเวอร์ โดยการอนุญาตให้มีกระบวนการพิสูจน์ตัวตนกับการใช้งานลายเซ็นดิจิทัลสำหรับการรักษาความถูกต้องของข้อมูลและการเข้ารหัสข้อมูลเพื่อป้องกันความเป็นส่วนตัวระหว่างการสื่อสารข้อมูล

SSL นั้นได้รับการยอมรับอย่างกว้างขวางบน World Wide Web ในการใช้สำหรับตรวจสอบและเข้ารหัสลับการติดต่อสื่อสารระหว่าง client และ server หน้าที่ของ SSL จะแบ่งออกเป็น 3 ส่วนใหญ่ๆ คือ

1. การตรวจสอบ server ว่าเป็นตัวจริง: ตัวโปรแกรม client ที่มีขีดความสามารถในการสื่อสารแบบ SSL จะสามารถตรวจสอบเครื่อง server ที่ตนกำลังจะไปเชื่อมต่อได้ว่า server นั้นเป็น server ตัวจริงหรือไม่ โดยใช้เทคนิคการเข้ารหัสแบบ public key ในการตรวจสอบใบรับรอง (certificate) และ public ID ของ server นั้น (โดยที่มีองค์กรที่ client เชื่อถือเป็นผู้ออกใบรับรองและ public ID ให้แก่ server นั้น)

หน้าที่นี้ของ SSL เป็นหน้าที่ที่สำคัญ โดยเฉพาะอย่างยิ่งในกรณีที่ client ต้องการที่จะส่งข้อมูลที่เป็นความลับ (เช่น หมายเลข credit card) ให้กับ server ซึ่ง client จะต้องตรวจสอบก่อนว่า server เป็นตัวจริงหรือไม่

2. การตรวจสอบว่า client เป็นตัวจริง: server ที่มีขีดความสามารถในการสื่อสารแบบ SSL จะใช้เทคนิคเช่นเดียวกับในหัวข้อที่แล้วในการตรวจสอบ client หรือผู้ใช้ว่าเป็นตัวจริงหรือไม่ โดยจะตรวจสอบใบรับรองและ public ID (ที่มีองค์กรที่ server เชื่อถือเป็นผู้ออกให้) ของ client หรือผู้ใช้นั้น

หน้าที่หนึ่งของ SSL จะมีประโยชน์ในกรณีเช่น ธนาคารต้องการที่จะส่งข้อมูลลับทางการเงินให้แก่ลูกค้าของตนผ่านทางเครือข่าย Internet (server ก็จะต้องตรวจสอบ client ก่อนว่าเป็น client นั้นจริง)

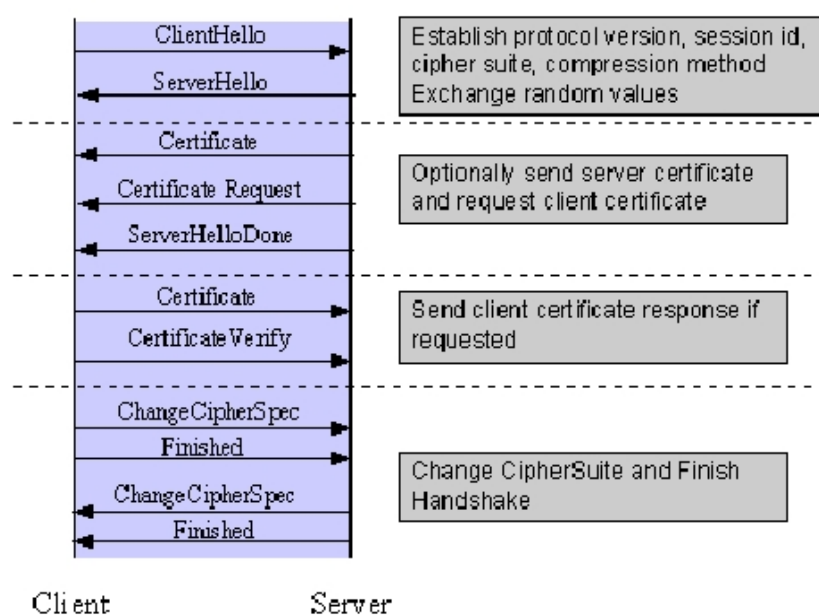
3. การเข้ารหัสลับการเชื่อมต่อ: ในกรณีนี้ ข้อมูลทั้งหมดที่ถูกส่งระหว่าง client และ server จะถูกเข้ารหัสลับ โดยโปรแกรมที่ส่งข้อมูลเป็นผู้เข้ารหัสและโปรแกรมที่รับข้อมูลเป็นผู้ถอดรหัส (โดยใช้วิธี public key) นอกจากการเข้ารหัสลับในลักษณะนี้แล้ว SSL ยังสามารถปกป้องความถูกต้องสมบูรณ์ของข้อมูลได้อีกด้วย กล่าวคือ ตัวโปรแกรมรับข้อมูลจะทราบได้หากข้อมูลถูกเปลี่ยนแปลงไปในขณะกำลังเดินทางจากผู้ส่งไปยังผู้รับ

โพรโตคอล SSL อนุญาตให้สามารถเลือกวิธีการในการเข้ารหัส วิธีสร้างไคเจสต์ [*1] และลายเซ็นดิจิทัล ได้อย่างอิสระก่อนการสื่อสารจะเริ่มต้นขึ้น ตามความต้องการของทั้งเว็บเซิร์ฟเวอร์และบราวเซอร์ ทั้งนี้เพื่อเพิ่มความยืดหยุ่นในการใช้งาน เปิดโอกาสให้ทดลองใช้วิธีการในการเข้ารหัสวิธีใหม่ รวมถึงลดปัญหาการส่งออกวิธีการเข้ารหัสไปประเทศที่ไม่อนุญาต

กระบวนการในการเริ่มต้นการสื่อสารผ่านชั้น SSL แบ่งเป็น 4 ขั้นตอนคือ

1. ประกาศชุดวิธีการเข้ารหัส ไคเจสต์ และลายเซ็นดิจิทัลที่สนับสนุนของทั้งไคลเอ็นต์และเซิร์ฟเวอร์
2. การพิสูจน์ตัวตนของเซิร์ฟเวอร์ต่อไคลเอ็นต์
3. การพิสูจน์ตัวตนของไคลเอ็นต์ต่อเซิร์ฟเวอร์ ถ้าจำเป็น
4. ไคลเอ็นต์และเซิร์ฟเวอร์ตกลงชุดวิธีการเข้ารหัส การสร้างไคเจสต์ และการใช้ลายเซ็นดิจิทัล

ตามรูป



รูปที่ 3.10 กระบวนการเริ่มต้นการติดต่อสื่อสารของโพรโตคอล SSL

ขั้นตอน 1: ประกาศชุดวิธีการเข้ารหัส โดเจสต์ และลายเซ็นดิจิทัลที่สนับสนุนของทั้งไคลเอ็นต์และเซิร์ฟเวอร์

ไคลเอ็นต์และเซิร์ฟเวอร์ส่งข้อความเริ่มต้นการสื่อสาร (Hello message) ซึ่งประกอบไปด้วยเวอร์ชันของโพรโทคอลที่ใช้ วิธีการเข้ารหัสที่เว็บเซิร์ฟเวอร์และไคลเอ็นต์สนับสนุน หมายเลขระบุการสื่อสาร (Session identifier) รวมถึงวิธีการบีบอัดข้อมูลในการสื่อสารที่สนับสนุน

หมายเลขระบุการสื่อสารที่เกิดขึ้น ใช้สำหรับตรวจสอบการเชื่อมต่อระหว่างไคลเอ็นต์และเซิร์ฟเวอร์ ถ้ามีการเชื่อมต่อก่อนหน้านี้เกิดขึ้น แสดงว่าได้มีการตกลงวิธีการสื่อสารแล้ว สามารถเริ่มต้นส่งข้อมูลได้ทันที เป็นการลดเวลาติดต่อสื่อสารลง

ขั้นตอน 2: การพิสูจน์ตัวตนของเซิร์ฟเวอร์ต่อไคลเอ็นต์

ถัดมาเว็บเซิร์ฟเวอร์ทำการส่ง Certificate หรือใบยืนยันความมีตัวตนของเซิร์ฟเวอร์ ไคลเอ็นต์จะทำการตรวจสอบ Certificate กับผู้ให้บริการ Certificate Authority ที่ได้ตั้งค่าไว้เพื่อยืนยันความถูกต้องของ Certificate ของเซิร์ฟเวอร์

ขั้นตอน 3: การพิสูจน์ตัวตนของไคลเอ็นต์ต่อเซิร์ฟเวอร์ ถ้าจำเป็น

เซิร์ฟเวอร์สามารถร้องขอ Certificate จากไคลเอ็นต์เพื่อตรวจสอบความถูกต้องของ Client ด้วยก็ได้ ใช้ในกรณีที่มีการจำกัดการใช้งานเฉพาะไคลเอ็นต์ที่ต้องการเท่านั้น ซึ่ง SSL สนับสนุนการตรวจสอบได้จากทั้งเซิร์ฟเวอร์และไคลเอ็นต์ ขึ้นอยู่กับการเลือกใช้งานในขณะติดต่อสื่อสารที่เกิดขึ้นนั้น

ขั้นตอน 4: ไคลเอ็นต์และเซิร์ฟเวอร์ตกลงชุดวิธีการเข้ารหัส การสร้างโดเจสต์ และการใช้ลายเซ็นดิจิทัล

ขั้นตอนการตรวจสอบ Certificate ที่เซิร์ฟเวอร์ร้องขอจากไคลเอ็นต์จะมีหรือไม่มีก็ได้ ขึ้นอยู่กับการตั้งค่าบนเซิร์ฟเวอร์ หลังจากขั้นตอนการตรวจสอบเสร็จสิ้น เซิร์ฟเวอร์และไคลเอ็นต์จะตกลงการใช้งานวิธีการเข้ารหัสระหว่างกันโดยใช้ค่าที่ได้จากการประกาศในขั้นตอนแรก

วิธีการแลกเปลี่ยนกุญแจในการเข้ารหัส (Key exchange method) คือการกำหนดกลไกการแลกเปลี่ยนกุญแจที่ใช้ในการเข้ารหัสระหว่างการสื่อสาร โดยทั้งไคลเอ็นต์และเซิร์ฟเวอร์จะใช้กุญแจนี้ในการเข้ารหัสและถอดรหัสข้อมูล ใน SSL เวอร์ชัน 2.0 จะสนับสนุนวิธีการแลกเปลี่ยนกุญแจแบบ RSA ส่วน SSL เวอร์ชัน 3.0 ขึ้นไปจะสนับสนุนวิธีการอื่นๆ เพิ่มเติมเช่นการใช้ RSA ร่วมกับการใช้ Certificate หรือ Diffie-Hellman เป็นต้น

วิธีการเข้ารหัสในปัจจุบันแบ่งเป็นสองวิธีคือ การใช้กุญแจเดียวกันในการเข้ารหัสและถอดรหัส อาจเรียกกุญแจนี้ว่า Session key หรือ Secret key ส่วนอีกวิธีคือการใช้กุญแจคนละตัวในการเข้ารหัสและถอดรหัส ประกอบไปด้วยกุญแจสาธารณะและกุญแจส่วนตัวซึ่งเป็นคู่กันเสมอ

เข้ารหัสด้วยกุญแจใด จะต้องถอดรหัสด้วยกุญแจที่คู่กันและตรงกันข้ามเท่านั้น มักใช้วิธีการเข้ารหัสด้วยกุญแจคนละตัวมาใช้ในการเข้ารหัส Session key และส่งไปให้ฝั่งตรงข้ามก่อนการสื่อสารจะเกิดขึ้นรวมเรียกว่าวิธีการแลกเปลี่ยนกุญแจในการเข้ารหัส

SSL ใช้วิธีการเข้ารหัสด้วยกุญแจสมมาตร หรือกุญแจเดียวในการเข้ารหัสและถอดรหัสตามที่กล่าวข้างต้น วิธีการเข้ารหัสคือ การเข้ารหัสด้วย DES และ 3DES (Data Encryption Standard), วิธีการเข้ารหัสด้วย IDEA ส่วน RC2 และ RC4 เป็นวิธีการเข้ารหัสของ RSA รวมถึงวิธีการเข้ารหัสแบบ Fortezza สำหรับความยาวของการเข้ารหัสที่ใช้คือ 40 บิต, 96 บิต และ 128 บิต

การสร้าง Message Authentication Code (MAC) เพื่อใช้สำหรับการยืนยันความถูกต้องของข้อมูลระหว่างการสื่อสารและป้องกันการปลอมข้อมูล ส่วนฟังก์ชันสร้างไคเจสต์ที่ SSL สนับสนุนและเลือกใช้ได้ในปัจจุบันคือ MD5 ขนาด 128 บิต และ SHA-1 (Secure Hash Algorithm) ขนาด 160 บิต ซึ่งจะได้อัลกอริทึมที่ทั้งสองฝ่ายสนับสนุนและเหมาะสมซึ่งเป็นขั้นตอนสุดท้ายก่อนการสื่อสารที่มีการเข้ารหัสจะเริ่มขึ้น

3.6. รูปแบบของคีย์และการนำไปใช้

จากการศึกษาเกี่ยวกับความปลอดภัยของเว็บเซอร์วิสทั้งจาวาและคอตเน็ตพบว่า “คีย์” หรือ ใบรับรองอิเล็กทรอนิกส์ (Certificates) มีความสำคัญมากในทุกๆส่วนซึ่งการสร้างและการนำไปใช้ของคีย์แต่ละชนิดก็มีความแตกต่างกันอย่างเห็นได้ชัดดังอธิบายด้านล่างนี้

ตารางที่ 3.1 แสดง การเปรียบเทียบความสามารถของคีย์แต่ละตัว

Types of Certificate							
Extension	Type	Holds Multiple Certificates?	Holds Private Keys?	Password Required?	Microsoft	Java	Notes
*.p7b	PKCS#7	Yes	No	No	Yes	Yes	IE binary public key export. Can optionally contain multiple certs, e.g. a certificate chain. IBM's <u>Keyman</u> will create and manage this format of keyring.

n/a	PKCS#8	No	Yes	Yes	n/a	n/a	The private-key information syntax standard. This defines a method to store Private Key Information.
*.csr,.p10	PKCS#10	No	No	No	n/a	n/a	The certification request syntax standard. This describes a syntax for certification requests. Contains the public key, signed with the private key.
n/a	PKCS#11 / CryptoKi	Yes	Yes	Yes	n/a	n/a	The cryptographic token interface standard. This defines a technology independent programming interface for cryptographic devices such as smartcards.
.p12, .pfx	PKCS#12	Yes	Yes	Yes	Yes	Yes	The personal information exchange syntax standard. This describes a portable format for storage and transportation of user private keys, certificates etc. Where a developer stores his code signing keys. .keystore is not in this format.

*.pem	PEM	n/a	No	n/a	n/a	n/a	Privacy Encoded Mail format for sending certs embedded in email, typically SSL cert.
*.sst	n/a	n/a	Yes	n/a	Yes	n/a	Windows certificate store.
*.stl	n/a	n/a	No	n/a	Yes	n/a	Windows certificate trust list. Used my cryptext.dll.
*.usr	X.509/DER binary format	n/a	No	n/a	Yes	n/a	user certificates.
*.asc	PGP	n/a	No	n/a	n/a	n/a	Pretty Good Privacy.
*.ca	X.509/DER binary format	No	No	No	Yes	Yes	root certificates.
*.cer	X.509/DER binary format	No	No	No	Yes	Yes	Sun Java 1.3+ user certs.
*.cer	X.509/DER BASE64 encoded	No	No	No	Yes	Yes	Sun Java 1.3+ user certs.
*.crt	X.509/DER binary format	No	No	No	Yes	Yes	Thawte root certificates, Sun Java 1.3+ cacerts.
*.db	proprietary binary?	Yes	Yes	No	Yes	n/a	Netscape export of the entire set of keys. Contains multiple certs with private keys.
.keystore, cacerts.	JKS	Yes	Yes	Yes	No	Yes	Sun's keyring format. Can optionally include private key, authentication chain and friendly name. Sun never imports/exports the private key, though .keystore contains it.

3.7. Certificates

สำหรับค่า Public key encryption เป็นระบบที่มีความน่าเชื่อถือและจะมีกลไกในการยืนยันและกระจายค่า Public keys ซึ่งค่าที่ส่งนี้ Windows Server 2003 รองรับ PKI ใช้ Digital Certificates เพื่อยืนยันบุคคลนั้นอยู่ในองค์กร

การใช้ Public key จะต้องมีการบริหารที่เรียกว่า Certification Authority (CA) ซึ่ง CA นี้สามารถที่เป็นบริษัทที่แยกดูแลจากภายนอกก็ได้ โดยจะกำหนดเป็นชั้นๆของซอฟต์แวร์ในเครื่อง ซึ่งการเลือก CA เราต้องขึ้นอยู่กับองค์กรที่ติดต่อด้วย

การบรรจุการรับรองจาก CA สามารถทำได้ด้วยมือ และเราก็ส่งคำร้องขอที่แน่นอนไปที่ CA หรือใช้แบบอัตโนมัติ คือระบบจะทำงานเป็นแบบกราวด์ในการทำงานแบบปกติ ซึ่ง CA จะรับ Public key และ Private key ว่าตรงกันหรือไม่ ซึ่งค่าที่เป็น Private จะเข้ารหัสเก็บไว้ในเครื่อง ผู้ใช้ตามแบบที่ถูกเข้ารหัส และค่า Public key จะอยู่ใส่ส่วนของ Certificate หรือผู้รับรอง โดยค่า Public key นี้จะใช้ในการกระจายให้กับบุคคลอื่นๆที่ต้องการ

3.7.1. ส่วนประกอบของ Certificate

Certificate คือ key ที่ใช้ในระบบ security ซึ่งมีส่วนประกอบดังนี้

Public key

เป็นตัวเลขที่เกี่ยวข้องกันซึ่งเป็นที่รู้กันว่ามีความน่าเชื่อถือในการติดต่อกันระหว่าง entity นิยมใช้คล้ายกับการ verify signature

Digitally Signed

ถ้ามีการใช้ digitally signed กับ entity ใดและมี signature ที่พิสูจน์ข้อมูลนั้น การนำข้อมูลนั้นไปใช้จะต้องมีการใช้ private key เพื่อถอด

Identity

ที่อยู่แน่ชัดของ entity ในบางครั้ง entity ก็คือ public key นั่นเอง

Signature

ข้อมูลที่ถูกเข้ารหัสด้วย private key

Private key

เป็นชุดของตัวเลขที่รู้เฉพาะผู้ที่ป็นเจ้าของ โดยจะมากู้กับ public key เสมอ

Entity

คือบุคคล , องค์กร , โปรแกรมคอมพิวเตอร์ , องค์กรธุรกิจ , ธนาคาร หรือสิ่งอื่นๆที่คุณสามารถอ้างถึงได้

3.7.2. การใช้ CA ภายในหรือภายนอก

เราสามารถวางระบบทั้งการใช้ CA ทั้งภายใน และภายนอก ซึ่งถ้าใช้ภายในเราต้องตั้ง PKI ในเครือข่าย หรือจะใช้ CA ของค่ายอื่นก็ได้ หรือใช้ทั้งคู่ ซึ่งการกำหนดจะต้องตรวจสอบกลุ่มที่ติดต่อว่าอยู่ในบริษัทเดียว หรือต่างบริษัท

ถ้าเราต้องการให้มีความปลอดภัยในองค์กรก็ตั้ง CA ขึ้นโดยใช้ Certificate Services ใน Windows Server 2003 ทำเป็น CA และให้ผู้ใช้ทั้งหมดทำการติดต่อใน CA ภายในนี้ การติดต่อข้ามที่อื่นทำไม่ได้เนื่องจากต้องการตรวจสอบ และระบุจาก CA

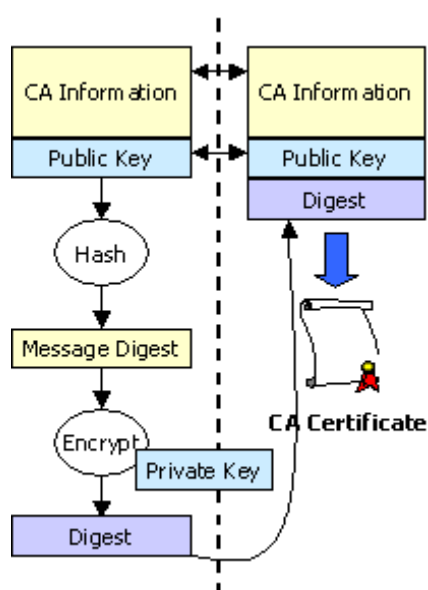
สำหรับการติดต่อภายนอกเราสามารถที่จะใช้ Commercial CA ที่ติดต่อเช่น Thawe และ VeriSign Inc

3.7.3. Certification Authority (CA)

คือหน่วยผู้ประกอบการรับรอง ที่เป็นหน่วยงานภายนอก (Trusted Third Party) ซึ่งมีหน้าที่หลักคือ เป็นผู้ออกใบรับรองอิเล็กทรอนิกส์ให้กับบุคคลหรือสิ่งอื่นๆ (ในที่นี้จะเรียกรวมว่าเป็น Subject) ทั้งนี้ Self-signed Certificate จะเป็นสิ่งที่ระบุตัวตนของหน่วยผู้ประกอบการรับรองเอง ดังนั้นหน่วยผู้ประกอบการรับรองจึงเป็นจุดเริ่มต้นของการสร้างความไว้วางใจ (Trust) แต่ละเอ็นทิตีจะต้องมอบความไว้วางใจต่อหน่วยผู้ประกอบการรับรองและสิ่งใดก็ตามที่หน่วยผู้ประกอบการรับรองได้ออกใบรับรองให้

3.7.4. การสร้าง Self-signed Certificate

ในการเริ่มต้นเป็นหน่วยผู้ประกอบการรับรอง จะต้องสร้างใบรับรองอิเล็กทรอนิกส์ของ CA เองก่อน ทั้งนี้ขั้นตอนการสร้าง Self-signed Certificate ของ CA เป็นดังในรูปที่ 3.11



รูปที่ 3.11 แสดงกระบวนการสร้าง Self-signed Certificate

หน่วยผู้ประกอบการรับรองเป็นหัวใจสำคัญของ PKI และรับผิดชอบหน้าที่สำคัญหลายประการ ได้แก่ Certificate Management กล่าวคือ Issuance, Revocation, Update และ Renewal การเผยแพร่ใบรับรองอิเล็กทรอนิกส์และ CRL (Certificate Revocation List) และทำการบันทึกที่ถือออกจากเหตุการณ์ต่างๆ ที่เกิดขึ้นในระบบ ขั้นตอนโดยทั่วไปในการออกใบรับรองให้กับ Subject ใดๆ นั้นจะประกอบไปด้วย

1. Subject ส่งคำร้องขอ (Certificate Request)
2. Subject เป็นผู้สร้างกุญแจคู่โดยหน่วยผู้ประกอบการรับรองเป็นผู้จัดเตรียมฟังก์ชันนี้ให้
3. หน่วยผู้ประกอบการรับรองจะจัดการออกใบรับรองให้ ทั้งนี้ คำร้องดังกล่าว อาจเป็นไปได้ตั้งแต่ขอใหม่ การปรับเปลี่ยนหรือขอให้ออกให้ใหม่
4. รอการอนุมัติจากหน่วยผู้ประกอบการรับรอง
5. หน่วยผู้ประกอบการรับรองตรวจสอบ Identity ของผู้ร้องขอ เมื่อผ่านแล้ว หน่วยผู้ประกอบการรับรองจะทำการ Sign Request นั้นโดยการสร้างใบรับรองอิเล็กทรอนิกส์
6. เผยแพร่ใบรับรองอิเล็กทรอนิกส์ดังกล่าว เพื่อให้ผู้ร้องขอมารับไป

3.7.5. Registration Authority (RA)

คือ หน่วยรับผิดชอบในการดำเนินงานจริง หน่วยผู้ประกอบการรับรองบางแห่งอาจจะแบ่งหน้าที่บางอย่างให้กับ RA ใน RFC 2510 Internet Public Key Infrastructure Certificate Management Protocols อ้างถึงหน้าที่ของ RA ไว้ว่า คือ การตรวจสอบความถูกต้องของบุคคล (Personal Authentication) การแจกจ่าย Token (Token distribution) รายงานการเพิกถอน ใบรับรอง (Revocation reporting) การกำหนดชื่อ (Name assignment) การสร้างกุญแจคู่ (Key generation) การจัดเก็บกุญแจคู่ (Archival of key pairs) เป็นต้น ในทางปฏิบัติแล้ว ส่วนใหญ่ RA จะทำหน้าที่ตรวจสอบความถูกต้องของบุคคลหรือ Subject ใดๆ ระหว่างกระบวนการออกใบรับรองอิเล็กทรอนิกส์ (Certificate Enrollment Process)

3.7.6. Certificate Management Protocol (CMP)

คือ ขั้นตอนหรือโพรโตคอลในการจัดการใบรับรองอิเล็กทรอนิกส์ โดยที่ Certificate Management Protocol หลายแบบได้ระบุถึงขั้นตอน Certificate Enrollment ในบางกรณีก็ระบุถึงขั้นตอน Certificate Revocation Process เวนเดอร์บางแห่งก็เพิ่มเติมฟังก์ชันการทำงานของโพรโตคอลนอกเหนือจากนี้เช่น Key recovery และ Automated certificate renewal Public Key Cryptographic Standard (PKCS) เป็นชุดโพรโตคอลมาตรฐานของ RSA Security ที่ระบุวิธีการรักษาความปลอดภัยของการแลกเปลี่ยนข้อมูลระหว่างกัน PKCS #1, #3, #5, #6, #7, #8, #9, #10,

#11, #12 และ #15 เป็นมาตรฐานที่มีการใช้งานในปัจจุบัน ส่วน PKCS #13 และ #14 ยังคงอยู่ในระหว่างการอนุมัติ ส่วน PKCS #2 และ #4 ได้ถูกรวมเข้ากับ PKCS #1 แล้ว

PKCS #1 : RSA Cryptography Standard

PKCS #2 : ถูกรวมเข้ากับ PKCS #1 แล้ว

PKCS #3 : Diffie-Hellman Key Agreement Standard

PKCS #4 : ถูกรวมเข้ากับ PKCS #1 แล้ว

PKCS #5 : Password-Based Cryptography Standard

PKCS #6 : Extended-Certificate Syntax Standard

PKCS #7 : Cryptographic Message Syntax Standard

PKCS #8 : Private-Key Information Syntax Standard

PKCS #9 : Selected Attribute Types

PKCS #10 : Certification Request Syntax Standard

PKCS #11 : Cryptographic Token Interface Standard

PKCS #12 : Personal Information Exchange Syntax Standard

PKCS #13 : Elliptic Curve Cryptography Standard

PKCS #15 : Cryptographic Token Information Format Standard

ในปี 1999 IETF ได้อนุมัติ RFC 2510 Internet Public Key Infrastructure Certificate Management Protocols และ RFC 2511 Internet Public Key Infrastructure Certificate Request Management Framework (CRMF) จากนั้น PKIX Working Group ได้พัฒนา Certificate Management Protocol (CMP) บนพื้นฐานจากมาตรฐานเหล่านี้ โดย CMP เป็นอีกทางเลือกหนึ่ง นอกเหนือไปจาก PKCS สำหรับจัดการ Certificate Request และ Response Message ทั้งนี้ CMP รองรับฟังก์ชันการทำงานของจัดการใบรับรอง หลากหลายกว่า เช่น Cross-Certification Request และ Response Message, Revocation Request และ Response Message, Certificate and CRL Distribution Message แต่ CMP เป็นโพรโตคอลที่ค่อนข้างซับซ้อน ที่แม้ซอฟต์แวร์ CA จะรองรับแต่ โปรแกรมต่างๆ หรืออุปกรณ์ก็ยังไม่รองรับเป็นที่แพร่หลายนัก

3.7.7. Certificate Revocation

คือกระบวนการเพิกถอนใบรับรองอิเล็กทรอนิกส์ (Certificate Revocation) เนื่องจากหมดอายุหรือใบรับรองอิเล็กทรอนิกส์ถูกผู้อื่นนำไปใช้ วิธีที่นิยมใช้ได้แก่

CRL (Certificate Revocation List) เป็นรายชื่อของใบรับรองที่ถูกเพิกถอน ใน RFC 2459 Internet X.509 Public Key Infrastructure Certificate and CRL Profile ได้ระบุรูปแบบ ของ X.509 CRL Version 2 ไว้

OCSP (Online Certificate Status Protocol) ในการใช้งาน PKI กับงานบางประเภท ช่วงเวลาระหว่างการอัปเดต CRL แต่ละครั้งไม่เป็นที่ยอมรับ ดังนั้นจึงเกิด OCSP ขึ้น ซึ่งเป็นกลไกในการตรวจสอบการเพิกถอนของใบรับรองอิเล็กทรอนิกส์แบบเรียลไทม์ เ็นที่ติจะส่ง Certificate Status Check ไปยัง OCSP Responder ที่อาจเป็นหน่วยผู้ประกอบการรับรองหรือ แบ่งหน้าที่ให้กับหน่วยอื่นก็ได้ ทั้งนี้ใบรับรองอิเล็กทรอนิกส์จะไม่ได้รับการยอมรับจนกว่า OCSP Responder จะตอบMessageมาขึ้นชั้นหรือปฏิเสธสถานะของใบรับรองอิเล็กทรอนิกส์นั้น

3.7.8. Certificate Repository

ใช้สำหรับจัดเก็บและประกาศใบรับรองอิเล็กทรอนิกส์ให้สาธารณะรับทราบ ในการพัฒนาระบบเล็กๆ ส่วนนี้อาจจะไม่จำเป็น เพราะอาจทำการแลกเปลี่ยนใบรับรองหรือ CRL ผ่านทางจดหมายอิเล็กทรอนิกส์ได้ แต่เมื่อต้องเกี่ยวข้องกับใบรับรองจำนวนมาก ก็จำเป็นต้องมีการจัดเก็บให้เป็นไปในทิศทางเดียวกัน ซึ่งอาจจะเลือกใช้ Directory Services หรือ FTP และ HTTP ก็ได้

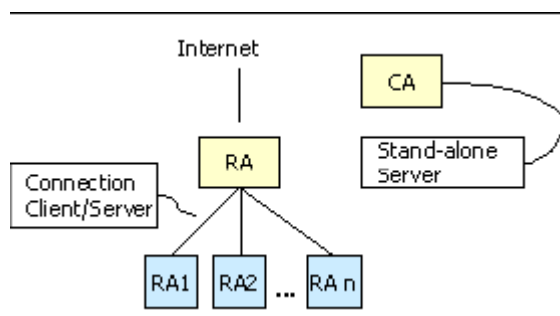
3.7.9. Time Stamp Authority (TSAs)

เนื่องจากองค์ประกอบเรื่องความปลอดภัยข้อหนึ่งคือ Non-repudiation (ความแน่ใจว่าคู่สื่อสารไม่สามารถที่จะปฏิเสธการกระทำทางอิเล็กทรอนิกส์ที่เกิดขึ้นได้) เรื่องของเวลาจึงเป็นส่วนประกอบสำคัญส่วนหนึ่ง ยกตัวอย่างการสื่อสารผ่านจดหมายอิเล็กทรอนิกส์ หากมีการใช้ Time Stamp Protocol (TSP) เช่น หากนาย ก ต้องการส่งจดหมายอิเล็กทรอนิกส์ให้นาย ข โดยแนบลายมือชื่อดิจิทัลไปด้วย นาย ก จะส่งไปยัง Time Stamping Authority (TSA) ก่อนเพื่อทำการ Sign Request ของนาย ก จากนั้น TSA ส่งกลับคืนนาย ก แล้วนาย ก จึงส่งจดหมายอิเล็กทรอนิกส์ให้นาย ข ทั้งนี้ หากนาย ข ปฏิเสธว่าไม่ได้รับจดหมาย นาย ก สามารถนำ Response จาก TSA มาอ้างอิงภายหลังได้ แต่ทั้งสองคนจะต้องให้ความไว้วางใจต่อ TSA ล่วงหน้าก่อนแล้ว

3.7.10. แผนโครงสร้างของ OpenCA

ส่วนที่เป็น CA จะแยกออกมาเป็นเซิร์ฟเวอร์ที่ไม่เชื่อมต่อกับเซิร์ฟเวอร์อื่นหรืออินเทอร์เน็ต เนื่องจากต้องการความปลอดภัยสูงเพราะเป็นเซิร์ฟเวอร์ที่ติดตั้ง CA Software และเก็บกุญแจส่วนตัวของ CA ไว้ด้วยเซิร์ฟเวอร์ RA จะมีความซับซ้อนมากกว่า กล่าวคือ จะมีเว็บเซิร์ฟเวอร์และเซิร์ฟเวอร์ LDAP รันอยู่ โดยอย่างแรกจะทำหน้าที่รองรับ Request จาก Subject ใดๆ ในการขอใบรับรองอิเล็กทรอนิกส์ ก่อนที่ RA จะเป็นผู้นำไปให้ CA รับรองต่อ ส่วนอย่างหลัง จะทำหน้าที่เป็นที่เก็บและเผยแพร่ใบรับรองอิเล็กทรอนิกส์ จากรูปที่ 3.12 จะพบว่า RA สามารถมีได้หลายแห่ง ทั้งนี้เพื่อ

แบ่งเบาภาระนั่นเอง

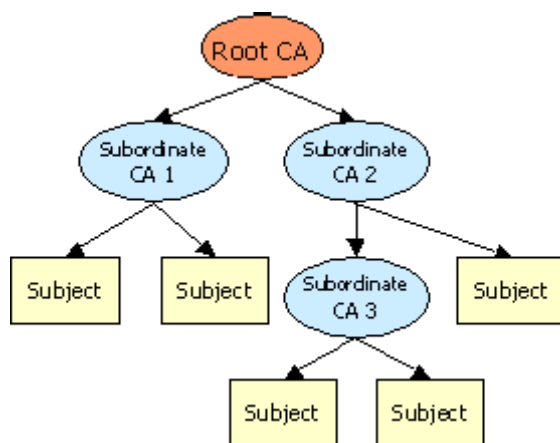


รูปที่ 3.12 แสดงโครงสร้างของ OpenCA

3.7.11. แผนโมเดลการให้ความเชื่อถือ (Trust Model)

Hierarchy

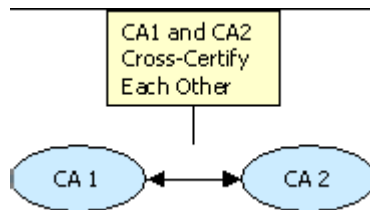
ในการสร้างหน่วยผู้ประกอบการรับรองนั้น หากเป็นการสร้างเพื่อใช้กันภายในองค์กรเอง แล้ว CA นั้นก็ย่อมจะได้รับความไว้วางใจจากบุคลากรในองค์กร แต่หากบุคคลในองค์กรต้องการที่จะติดต่อสื่อสารกับบุคคลภายนอกโดยใช้ใบรับรองอิเล็กทรอนิกส์ที่ออกให้โดยองค์กรของตน บุคคลภายนอกอาจไม่รู้จักและไม่ให้ความไว้วางใจ CA ภายใน บางครั้ง การรับรองคู่กุญแจที่สร้างเองก็ไม่ใช่ที่ยอมรับ จำเป็นต้องขอให้ CA อื่นที่เป็นรู้จักและได้รับความไว้วางใจมากกว่ารับรองให้ ดังในรูปที่ 3.13 Root CA จะเป็นผู้รับรองให้ Subordinate CA1 และ 2 ใครก็ตามที่มอบความไว้วางใจให้กับ Root CA ก็จะถือว่ามอบให้ Subordinate CA1 และ 2 ด้วยเช่นกัน เรียกว่าเป็น Trust Model แบบ Hierarchy (แบบลำดับ)



รูปที่ 3.13 แสดง Trust Model แบบ Hierarchy

Cross-Certification

โมเดลการมอบความไว้วางใจดังในรูปที่ 3.14 นั้นเรียกว่า Cross Certification หน่วยผู้ประกอบการรับรองจะผลัดกัน Sign ใบรับรองของ CA ให้แก่กัน ผู้ที่ได้ใบรับรองจากไม่ว่าจะ CA1 หรือ CA2 ก็จะมอบความไว้วางใจให้กับทั้ง 2 ที่ ทั้งนี้ เป็นเรื่องยากที่จะทำให้ CA แต่ละแห่งมอบความไว้วางใจให้แก่กัน จึงไม่เป็นที่นิยมนัก ผู้ใช้ที่ใช้บริการจาก CA คนละแห่งจึงต้องมีหน้าที่เลือกมอบความไว้วางใจให้กับ CA ใดๆ เอง



รูปที่ 3.14 แสดง Trust Model แบบ Cross-Certification

โมเดลการมอบความไว้วางใจนั้น จะมีการออกแบบและใช้งานก็ต่อเมื่อมีหน่วยผู้ประกอบการรับรองมากกว่า 1 แห่งขึ้นไป ดังนั้น การทดสอบใช้งานซอฟต์แวร์ในบทความนี้เป็น การออกแบบหน่วยผู้ประกอบการรับรองที่ใช้งานภายในเพียงหน่วยเดียว จึงยังมิได้กำหนดโมเดลการมอบความไว้วางใจกันไว้

3.7.12. เข้าใจหน้าที่ของ PKI

ใน Windows Server 2003 PKI จะมีงานดังนี้

- Publish certificates สามารถที่จะสร้าง การรับรองบน Web Site หรือ Active Directory ที่ผู้ใช้ เครื่อง แอปพลิเคชัน
- Enroll Clients ลงทะเบียนจากลูกค้าเข้ามาที่ CA โดยจะได้ชื่อลูกค้า
- User Certificates เมื่อลูกค้าร้องขอ และรับการรับรอง สามารถที่สื่อสารได้อย่างปลอดภัย และสามารถที่ทำหน้าที่กับรายการที่ออกให้
- Renew certificates เป็นการต่ออายุเมื่อครบ
- Revoke certificates เป็นการถอนคำที่รับรองแล้วไปเพิ่มในใน Certificate revocation list (CRL)

3.8. การออกแบบ Public Key Infrastructure

ส่วนประกอบของเครือข่ายส่วนมากคือการวางระบบ Public key infrastructure ซึ่งต้องมีการวางระบบอย่างระมัดระวัง โดยมีขั้นตอนดังนี้

1. กำหนดความต้องการ Certificate
2. การสร้างโครงสร้างพื้นฐาน Certification
3. การกำหนดค่า Certification

3.8.1. กำหนดความต้องการ Certificate

ในเฟสนี้ต้องมีการออกแบบเครือข่าย และกำหนดลูกข่ายที่มีความจำเป็นที่ต้องใช้ และบอกถึงว่า Certificate ช่วยอะไรในการทำงาน ชนิดการรับรองที่ลูกข่ายจำเป็น รวมถึงการตอบคำถามเกี่ยวกับระบบความปลอดภัย และยุทธศาสตร์ในการดำเนินงานด้านความปลอดภัย PKI ใช้กับเครื่อง Windows Server 2003 สามารถที่สร้าง และรองรับสิ่งต่างๆดังนี้

- Digital Signatures รองรับลายเซ็นอิเล็กทรอนิกส์
- การเข้ารหัสระบบไฟล์ผู้ใช้ และการซ่อมคืนการรับรอง คือใช้กับ EFS ใน Windows Server 2003 และการกำหนด Recovery agents หรือกลุ่มที่สามารถที่จะนำไฟล์กลับคืนมาให้ได้ โดยกลไกทั้งหมดใช้ PKI ในการจัดการ
- Internet authentication สามารถที่ใช้สร้างการเชื่อมต่อที่ปลอดภัยในการติดต่อระหว่างคู่ Server กับลูกข่าย
- IP Security หรือ IPSec สามารถที่ใช้กลไกนี้ในการเข้ารหัสสปีด และใช้กับการป้องกันข้อมูลในการโอนถ่าย
- Secure e-mail เข้ารหัสอิเล็กทรอนิกส์
- Smart Card logon เป็นการใช้กับวงจรด้านการรับรองด้วยบัตร ด้วยการเก็บ Smart card เป็น Private key เพื่อเข้าใช้เครือข่าย
- Software code signing เป็นการใช้กับ Microsoft's Authenticode technology ที่ใช้ในการดาวน์โหลด และติดตั้งโดยตรงจากผู้ประกาศโดยไม่มีการแก้ไข
- Wireless network authentication เพิ่มความปลอดภัยใน Wireless LAN ที่อาจมีการแอบดักฟังได้สิ่งที่จะนำแอปพลิเคชันใดมาใช้ต้องพิจารณาถึงความต้องการความปลอดภัยแค่ไหนในกลุ่มคนใด ซึ่งเราจะต้องพิจารณาร่วมกับการทำงานเช่น
- ถ้ามีการต่ออินเทอร์เน็ตจึงใช้ซอฟต์แวร์ที่เกี่ยวข้องกับอินเทอร์เน็ตได้
- ความปลอดภัยแบ่งเป็น 4 ระดับคือ พื้นฐาน กลาง สูง และภายนอก
 - ทั่วไปองค์กรเราต้องการใช้งานด้าน e-mail, EFS
 - ป้องกันข้อมูลระดับกลาง เช่นป้องกันข้อมูลที่อ่อนไหวด้วย IPSec
 - ต้องการมีความปลอดภัยสูงด้วย Smart card
 - มีการติดต่อภายนอกพิจารณาเรื่อง Internet authentication

สรุปได้ดังตาราง

ตารางที่ 3.2 แสดง การเปรียบเทียบระดับความปลอดภัย

Basic security	Medium Security	High Security	External Security
Secure e- mail	Secure e- mail	Secure e-mail	Software code signing
EFS	EFS	EFS	Internet authentication
	IPSec	IPSec	
		Smart Card	

3.8.2. การสร้างโครงสร้างพื้นฐาน Certification

หนึ่งในการพิจารณาที่เราต้องดูว่าเมื่อเรานำเข้ามาแล้วจะดำเนินการอย่างไร มีการออกแบบเป็นระดับชั้นหรือไม่ เช่นมี Root CA และมี Intermediate > Issuing ระดับติดต่อผู้ใช้ย่อยลงมาเมื่อสร้างโครงสร้างพื้นฐาน CA แล้วเราก็จะตรวจสอบตำแหน่ง และความสัมพันธ์ของ CA ต่างๆการใช้ CAs ภายใน หรือภายนอกอย่างที่เราสามารถที่ตั้ง CA เอง หรือจะใช้ Commercial CA ที่มีอยู่ได้ เราสรุปได้ดังนี้

ตารางที่ 3.3 แสดง การเปรียบเทียบระหว่าง CA ภายในกับ CA ภายนอก

ข้อดีของ CA ภายใน	ข้อเสีย CA ภายใน	ข้อดี CA ภายนอก	ข้อเสีย CA ภายนอก
ควบคุมโดยตรง	เสียแรงในการบริหารจัดการการรับรอง	เพิ่มความมั่นใจกับลูกค้าต้นทุนเพิ่มในองค์กร	
ไม่มีค่าใช้จ่ายเพิ่ม	นานวันยิ่งวุ่นวายในการกระจาย	รองรับ สิ่ง ที่ เกิด ขึ้น สำหรับ PKI มีปัญหา	ไม่สามารถลงทะเบียนอัตโนมัติ
สามารถใช้กับ AD	องค์กรต้องรับ ความรับผิดชอบเมื่อ PKI มีปัญหา	ได้รับคำแนะนำจากช่อง มีที่มีประสบการณ์ และงานด้านกฎหมาย	ไม่ยืดหยุ่นในการจัดการ
อนุญาตให้เพิ่ม PKI ต้นทุนต่ำ	จำกัดกับการติดต่อลูกค้าภายนอก	ลดต้นทุนการบริหาร	จำกัดในการเข้าใช้กับโครงสร้างในองค์กร

ในหลายครั้งเราจะใช้ทั้งสองกรณีคือถ้าเป็นการติดต่อภายในก็จะใช้ CA ภายใน แต่ถ้าต้องการเข้ารหัสภายนอกก็จะใช้ CA ภายนอก

การตั้ง CA สิ่งที่พิจารณาหลักคือเราต้องเลือกที่จะลงทะเลเบียน โดยหนึ่ง CA ของ Windows Server 2003 รองรับการใช้งานได้ถึง 35 ล้านรายการ หรือจะรองรับได้ 2 ล้าน หรือมากกว่าต่อวัน ซึ่งถ้าเรามีหลาย CAs ก็จะเพิ่มจำนวนการร้องขอ และใช้งาน สิ่งที่พิจารณามีดังนี้

- จำนวน และความเร็วของ Processors
- ความยาวคีย์
- ประสิทธิภาพดิสก์

ซึ่งจากพื้นฐานต่างๆเหล่านี้ทำให้เราสามารถที่นำไปออกแบบการติดตั้ง CA Server ได้ และถ้าต้องการที่รองรับ Fault tolerance ก็จะต้องมี RAID และถ้าต้องการลดการจราจรใน WAN ก็จะต้องมีการกำหนดการลงทะเลเบียนที่ท้องถิ่น

3.8.2.1. การสร้าง CA เป็นลำดับชั้น

เราสามารถที่กำหนดความสัมพันธ์ Parent/Child ในการตั้ง CA ใน PKI ได้เป็น Root CA กับ Subordinate CA ใน Subordinate CA ถ้าอยู่ตรงกลางเรียกว่า Intermediate CA และจะติดต่อกับ Root CA อีกทอดหนึ่ง ทั้งหมดเรียกว่า Certificate chain เมื่อเราแสดงค่าที่กำหนดจะมีระดับในแท็บ Certification Path ในองค์กรที่มีการใช้ PKI จำนวนมากก็จะมีโครงสร้างแต่ละระดับมาก หรือมีการกำหนดเป็นลำดับชั้น Root > Intermediate > Issuing CAs เป็นต้น

3.8.2.2. ชนิดของ Windows Server 2003 CA

ในระบบปฏิบัติการ Windows Server 2003 มีอยู่สองแบบคือ

- Enterprise เป็นแบบที่รองรับร่วมกับ Active Directory ซึ่งการทำงานลงทะเลเบียนต้องใช้ควบคู่กับ Active Directory ลูกข่ายต้องเข้าใช้ Active Directory ได้ ซึ่งไม่เหมาะกับลูกข่ายนอกองค์กร
- Stand-alone เป็นการที่ไม่ใช้ Active Directory แต่การทำงานทั้งหมดไม่อัตโนมัติต้องกำหนดเองทั้งหมด เหมาะกับการลงทะเลเบียนนอกองค์กรซึ่งถ้าเราวางโครง Enterprise หรือ stand-alone CA เราสามารถที่กำหนดเป็น Root หรือ Subordinate ได้ ซึ่งการกำหนดเหล่านี้ทำให้มีโครงสร้างเป็นลำดับชั้น

3.8.3. การกำหนดค่า Certification

เริ่มจากการออกแบบ การกำหนดค่าที่เป็นโครงสร้าง จะลำดับขั้นหรือไม่ก็ตาม เราต้องมีความเข้าใจในการกำหนดต่างๆดังนี้

- ชนิดของ Certificate เป็นการกำหนดหน้าที่ และการเก็บ Certificate templates ที่ให้กำหนดได้ง่ายกับชนิดที่ระบุในการรับรอง
- การเข้ารหัส และความยาวคีย์ รวมถึงอัลกอริทึม เป็นค่าที่ตรวจสอบ และกำหนดซึ่งมีผลต่อการทำงานของเครื่อง การเลือกระบบการเข้ารหัสหลายวิธีจะทำให้การทำงานช้าลง
- อายุของ Certificate เป็นการกำหนดให้ใช้งานและถ้ากำหนดสั้นไปก็ต้องกำหนดบ่อย มีผลต่อการจราจร
- นโยบายการต่ออายุ เป็นการกำหนดให้มีการต่ออายุ โดยสามารถใช้คีย์เดิมที่มีอยู่ได้ ซึ่งแนะนำให้ใช้คีย์ใหม่เพื่อความปลอดภัย แต่ก็มีผลกระทบต่อประสิทธิภาพ

3.9. การจัดการ Certificates

เมื่อเรามีการออกแบบโครงสร้าง PKI เรียบร้อยแล้วก็มาสู่ในเรื่องการบริหารงาน ซึ่งมีการลงทะเบียน การจัดการ และการประกาศ Certificate revocation list (CRL) เข้าใจเกี่ยวกับการลงทะเบียน และการต่ออายุ

CAs ที่มีลูกข่ายใช้งานอยู่ จะมีชนิดของ CAs ที่ติดตั้งดังนี้

- ถ้าติดตั้ง enterprise CAs ก็จะสามารถที่ลงทะเบียนได้โดยอัตโนมัติ ใน CAs เมื่อได้รับการร้องขอการรับรองจากลูกข่าย ก็จะประเมินการให้ใช้งาน หรือการปฏิเสธการใช้งาน
- ถ้าติดตั้ง Stand-alone CAs จะไม่สามารถลงทะเบียนได้อัตโนมัติ ต้องบริหาร หรือเฝ้าดูเอง ใน CA Console สำหรับการรับเข้า และปฏิเสธ

การใช้ลงทะเบียนอัตโนมัติ

เมื่อมีการกำหนดอนุญาตให้มีการร้องขอ และการรับการรับรองจาก CA ซึ่งทำงานอยู่บน Domain controllers ที่ทำงานบน Windows Server 2003 และลูกข่ายเป็น Windows XP Professional โดยมี Enterprise CA ทำงานอยู่ เราสามารถที่ควบคุมการลงทะเบียนได้โดยอัตโนมัติ รวมถึงการกำหนดใช้กับ Group Policy ใน Certificate templates

โดยดีฟอลท์ GPO จะกำหนดค่าลงทะเบียน User และ Computer อัตโนมัติในโดเมน ค่าที่กำหนด Windows Settings\Security Setting\Public Key Policies ทั้งใน User และ Computer configuration

ในกรณีที่ควบคุมการลงทะเบียนเราสามารถจัดการได้ใน Certificate Templates ซึ่งจะใช้กำหนดรอบ หรือชนิด ในการลงทะเบียน โดยกำหนดอนุญาตในการร้องขอการรับรองโดยใช้ Templates

เมื่อลูกค้าเข้ามาร้องขอก็จะตรวจสอบคุณสมบัติใน Active Directory ของแอปพลิเคชันลูกค้าว่ากำหนดให้อนุญาตหรือไม่ ถ้าได้รับก็จะลงรับรองให้โดยอัตโนมัติ

การลงทะเบียนด้วยมือ

ในเครื่อง Standalone CAs ไม่สามารถที่รับการลงทะเบียนอัตโนมัติ เราสามารถที่กำหนดการรับเข้ามา และต้องเฝ้าดูเพื่อที่จะให้อนุญาตผู้ใช้ที่ต้องการ ซึ่งค่าที่ร้องขออยู่ในโฟลเดอร์ Pending Requests เมื่อเราอนุญาตก็จะอยู่ในโฟลเดอร์ Issued Certificates

การจัดการร้องขอด้วยมือ

ในกรณีที่มีการร้องขอของ Certificate และรับค่าจากทั้ง Client และ Administrator เราสามารถใช้หนึ่งในเครื่องมือเหล่านี้ที่ให้มากับ Windows Server 2003

การใช้ Certificates Snap-in

เราสามารถที่ดูและจัดการการรับรองทั้งผู้ใช้ และเครื่องที่แสดงในโฟลเดอร์ และเราสามารถกำหนดให้ผู้ใช้ หรือเครื่องที่ต้องการได้ ค่าที่กำหนดใช้ Certificate Request Wizard และ Certificate Renewal Wizard

การใช้ Web Enrollment

เมื่อบริการ Certificate ทำงานบนเครื่อง Windows Server 2003 เราสามารถที่ลง IIS และกำหนดให้รองรับกับ ASP เพื่อที่จะสร้างเว็บเพจในการลงทะเบียน CA โดยใช้ผ่านเว็บ ซึ่งอาจจะรองรับการร้องขอจากที่ใดก็ได้ ตำแหน่งที่กำหนดอยู่ที่ http://IP_Address/Certsrv ถ้าเรากำหนดที่ Advanced ก็จะมีค่าชนิด Certificate ที่ให้กรอกในการลงทะเบียน

การยกเลิกการรับรอง

ในกรณีที่ผู้บริหารยกเลิกการรับรอง ค่าคีย์ Private ยังใช้อยู่ หรือผู้ใช้ที่ไม่มีสิทธิในการใช้ CA หรือต้องการเปลี่ยนค่าพารามิเตอร์ให้แตกต่าง เช่นคีย์ยาวขึ้น เราต้องทำการยกเลิก ซึ่งค่าที่เก็บอยู่ใน CRL

Enterprise CA จะประกาศ CRL ใน Active Directory database ลูกข่ายสามารถที่ใช้การสื่อสารกับ LDAP หรือถ้า Stand-alone CA ที่เก็บ CRL ลูกข่ายต้องใช้ Internet communication protocol เช่น HTTP หรือ FTP ทุกการรับรองจะเก็บ Path ที่กระจายใน CA สำหรับ CRLs เราสามารถที่แก้ไข path ได้ และตรวจสอบจุด CRL ที่ระบุใน Certificate ที่ทำให้มั่นใจว่าการรับรองไม่ได้ถูกยกเลิก ถ้า CRL ไม่มีอยู่ในจุดที่กระจาย แอปพลิเคชันก็จะปฏิเสธการรับรอง

การเลือกโฟลเดอร์ Revoked Certificates ใน CA Console และค่าที่แสดงมีการระบุกับ CRL ใหม่ และกำหนด CA ที่ประกาศ delta CRLs ซึ่งในรายการทั้งหมดที่ยกเลิกตั้งแต่ CRL ที่ประกาศสุดท้าย ในองค์กรขนาดใหญ่จะใช้ delta CRLs แทนที่ CRLs ที่เป็นการใช้งานกับแบนด์วิดท์เครือข่าย ซึ่ง delta CRLs ทำทุกสัปดาห์ และประกาศ base CRLs ทุกเดือน

3.10. KeyStore ในภาษาจาวา และ ดอทเน็ต

ในอดีตรหัส encrypt และ decrypt message จะใช้ private key และ public key ในการ encrypt และ decrypt message ดังตัวอย่างต่อไปนี้

- Alice กับ Bob ตกลงกันว่าจะใช้ public key algorithm ในการสื่อสาร
- Bob ส่ง public key ของเค้าให้ Alice
- Alice encrypt message ของเธอด้วย Bob public key แล้วส่งไปให้ Bob
- Bob นำ message มา decrypt ด้วย private key ของเขา

จากรูปแบบดังกล่าว public key นี้ยังไม่สามารถเก็บข้อมูลรายละเอียดของเจ้าของ key ได้ว่าเป็นของใคร ใช้ทำอะไร และอื่นๆ จึงมีการออกแบบรูปแบบอิเล็กทรอนิกส์แบบใหม่หรือที่เรียกว่า certificate ซึ่งภายในจะประกอบไปด้วยข้อมูลต่างๆ เช่น รายละเอียดของเจ้าของ key, public key และจุดประสงค์ในการใช้งาน โดยการสร้าง certificate ทำได้โดย

1. สร้าง public key และ private key ของผู้ใช้งานมาก่อน (ในการสร้าง key สามารถทำได้โดยหลาย algorithm)
2. สร้าง certificate signing request(CSR) ซึ่งประกอบด้วยรายละเอียดของเจ้าของคีย์, public key
3. นำ CSR ดังกล่าวส่งไปให้ certificate authority(CA) เป็นคน sign certificate ให้ โดย certificate จะถูกสร้างขึ้นและถูก sign ด้วย private key ของ CA
4. user ได้รับ certificate ดังกล่าวซึ่งพร้อมสามารถใช้งานได้

หลังจากวิธีดังกล่าวข้างต้น เราสามารถรู้ได้ว่า public key ที่เรานำมาใช้เป็นของใครและใช้เพื่ออะไร ตัวอย่างที่เห็นได้ชัดในการใช้ certificate เช่น SSL (secure socket layer) สำหรับการสร้าง certificate นั้นเราอาจจะไม่จำเป็นต้องให้ CA เป็นผู้ sign certificate ให้กับเราก็ได้ โดยเราสามารถที่จะใช้ private key ของเราเป็นตัว sign certificate ขึ้นมาเองได้ โดยวิธีนี้เรียกว่า self-sign

โดยปกติการสร้าง key entries ของภาษาจาวานั้น key entries ที่ได้จะถูกเก็บอยู่ใน KeyStore โดย KeyStore นี้เป็น interface ของภาษาจาวา ส่วน KeyStore implementations จะขึ้นอยู่กับ provider แต่ละ security provider เป็นผู้กำหนดว่าจะมี KeyStore ประเภทไหนบ้าง โดยปกติแล้ว sun provider ได้มีการ implement KeyStore ซึ่งเป็น Type JKS มากับ j2se (java 2 standard edition) สำหรับ provider package อื่นๆ ได้มีการ implement KeyStore ในรูปแบบอื่นๆไว้ โดยรูปแบบที่เป็นมาตรฐานรูปแบบหนึ่ง คือ PKCS12 ซึ่งสามารถหาใช้งานได้จาก provider ตัวอย่างเช่น provider package Bouncy Castle (www.bouncycastle.org) ซึ่งเป็น opensource โดยมี KeyStore เพิ่มเติมให้ใช้เช่น BKS, UBER, PKCS12 โครงสร้างของ key entries จะประกอบด้วย public key, private key และ certificate ถูกใส่ (chain certificate) นอกจากนี้ KeyStore จะเก็บ trusted certificate entries ซึ่งภายในจะเก็บ single public key certificate ของ CA สำหรับรูปแบบของ keystore แบบต่างๆ มีดังต่อไปนี้

- JKS เป็นรูปแบบ keystore ที่สามารถเก็บ private key และ certificate ซึ่งมี public key บรรจุอยู่ได้หลายชุด โดยใช้วิธีการเก็บแบบ map โดยมี alias เป็นตัวระบุชื่อ(ก็ใช้ในการ map) ส่วน component อื่นๆจะเป็น value สำหรับ alias นั้นๆ โดยแต่ละ component จะมี password ของก็ยซึ่งสามารถกำหนดได้ และจะต้องมี password ในระดับ KeyStore ด้วย
- PKCS12 (personal information exchange syntax standard) รูปแบบนี้เป็นรูปแบบที่นิยมใช้ทั่วไป ตัวอย่างแอปพลิเคชันที่รองรับรูปแบบนี้ เช่น certificate manager ของ windows, OpenSSL รูปแบบจะเก็บ 1 key component ต่อ 1 KeyStore เท่านั้น และแต่ละ KeyStore จะต้องมีการมี password เพื่อป้องกันการเข้าถึง private key
- BKS (Bouncy Castles's version ของ JKS) เป็น KeyStore ที่มีรูปแบบโครงสร้างเหมือนกับ JKS สามารถทำงานร่วมกับ keytool ได้ keystore ชนิดนี้จะป้องกันการแก้ไข แต่ไม่ป้องกันการตรวจสอบ
- UBER (Bouncy Castles's secure version ของ BKS) เป็น KeyStore ที่สามารถทำงานร่วมกับ keytool ได้เท่านั้นถ้า password ถูกกำหนดจาก command line เนื่องจาก key ทั้งหมดถูก encrypt โดย PBEWithSHAAndTwofish-CBC วิธีนี้จะทำให้ KeyStore สามารถสกัดกั้นการแก้ไขและตรวจสอบได้ และมีการบังคับให้มีการตรวจสอบเสมอ

- JCEKS เป็น KeyStore ของ SUN JCE provider เริ่มมีใน j2sdk version 1.4 เพื่อเพิ่มความสามารถในการป้องกัน private key ได้ดีกว่า KeyStore JKS ซึ่งเป็นของ SUN provider โดยใช้ password-based encryption with Triple DES ในการป้องกันแต่ละ key entries แต่ JKS ใช้เพียงแค่ hash value ในการป้องกันแต่ละ key entries ข้อแตกต่างจาก JKS อีกข้อหนึ่งคือ JCEKS มันจะเก็บ private key หรือ secret key อย่างใดอย่างหนึ่งไว้ใน key entries
- ถ้ามีการใช้ Java Cryptography Extensions(JCE) จะไม่มีการใช้ JKS ส่วน JCEKS จะใช้เมื่อมีการใช้ JCE ถึงแม้ว่า JCEKS จะมีความปลอดภัยสูงกว่าประสิทธิภาพของมันก็น้อยลงไปด้วย

รูปแบบของ key มีดังนี้ JKS ,JCEK,PKCS12,JCERACFKS(z/OS) ซึ่งรูปแบบ JKS จะไม่เก็บ shared key ส่วน JCEK เป็นรูปแบบที่มีความปลอดภัยสูง และรูปแบบ PKCS12 เป็นรูปแบบของ key มาตรฐาน

KeyStore ที่กล่าวมาข้างต้นที่นอกเหนือจาก JKS สามารถใช้กับ utility keytool ได้ทั้งหมด โดยจะต้องกำหนดพารามิเตอร์ -storetype, -providerName, -providerClass และ -providerArg ให้เหมาะสมกับแต่ละ keystore โดยสามารถหาข้อมูลเพิ่มเติมได้จาก security provider document สำหรับ keystore ที่จะใช้

การสร้าง KeyStore นั้น สามารถทำได้สองวิธี คือ การเขียนโปรแกรมโดยใช้ java.security package หรือใช้ keytool ซึ่งเป็น management utility ในการ generate public key และ private key ขึ้นมานั้นจะต้องมี key algorithm และ signature algorithm ซึ่งสองอย่างนี้จะขึ้นอยู่กับแต่ละ provider ว่ามี algorithm แบบไหนให้ใช้บ้าง โดยปกติแล้ว key algorithm ที่ใช้มี 2 ชนิด คือ RSA กับ DSA ที่นิยมใช้ ส่วน signature algorithm ถ้า key algorithm เป็น DSA มักจะใช้ signature algorithm เป็น SHA1withDSA ส่วน RSA มักจะใช้ signature algorithm เป็น MD5withRSA

สำหรับการสร้าง KeyStore แบบ JKS โดยใช้ keytool ทำได้ดังต่อไปนี้

```
keytool -genkey [-alias <alias>] [-keysize <keysize>] [-keyalg <keyalg>] [-keystore keystore]
```

<alias> คือ ชื่อของแต่ละ key component โดย ค่า default จะเป็น mykey

<keysize> คือ ขนาดของ key ที่ gen ขึ้นมา โดยถ้าเป็น DSA ใช้ได้แค่ 512 ถึง 1024 โดยต้องเป็นค่าที่หารด้วย 64 ลงตัว สำหรับ RSA สามารถตั้งแต่ 512 ถึง 4098

<keyalg> เป็นได้ทั้ง RSA และ DSA แต่ค่า default เป็น DSA

<keystore> เป็นการระบุ file KeyStore ที่สร้างขึ้นว่าอยู่ไหน โดยปกติแล้ว file ที่ถูกสร้างขึ้นจะถูกเก็บไว้ใน user directory ของแต่ละ operating system โดยมีชื่อ file ว่า .keystore ตัวอย่างเช่น

```
keytool -genkey -alias mykey -keyalg RSA -keysize 1024 -keystore keystore.jks
```

จากตัวอย่างเป็นการสร้าง key โดยให้ key มี alias เป็น mykey , key algorithm ที่ใช้เป็น RSA, key มีขนาด 1024 บิต และเก็บ keystore ที่ได้ไว้ในไฟล์ keystore.jks

สำหรับการสร้าง KeyStore แบบอื่นๆ สามารถใช้ keytool ได้ แต่ต้องมี provider class ของ key store แบบอื่นๆ ไปด้วย เช่น PKCS12 ซึ่งการสร้างแบบนี้ไม่นิยมใช้ keytool สร้าง แต่นิยมใช้การเขียน code gen ขึ้นมาเลย